# ahmed-infra
# The complete guide

An enterprise-style Linux administration lab on AWS: architecture, tools, every component, every decision, and how to run the company day to day

Company: **ahmed** (fictional) · Administrator: Ahmed
Platform: AWS us-east-1, RHEL 10, Terraform, Ansible, FreeIPA
State of the project: roadmap items 1 to 5 complete (app01 deferred)
Repository: github.com/ahmeddhussain/Linux-System-Administration
Written 2026-11-03

# Contents

**1. The project in one chapter ..... 4**
1.1 What this project is ..... 4
1.2 The company ..... 4
1.3 The six hosts ..... 4
1.4 The principles everything follows ..... 5
1.5 How the project was built (timeline) ..... 5
1.6 How to read this guide ..... 6

**2. Tools and technologies, explained from zero ..... 7**
2.1 AWS building blocks ..... 7
2.2 RHEL 10 and the operating-system tools ..... 8
2.3 Terraform ..... 9
2.4 Ansible ..... 9
2.5 FreeIPA (Red Hat Identity Management) ..... 10
2.6 Networking and remote access tools ..... 10
2.7 Logging, audit and security tools ..... 11
2.8 Backups and the AWS CLI ..... 11
2.9 Where each tool came from ..... 11

**3. Architecture ..... 12**
3.1 The network ..... 12
3.2 Who reaches what ..... 13
3.3 Defence in depth: the layers a connection must pass ..... 13
3.4 Where the data lives (and why) ..... 14
3.5 The three Terraform layers ..... 14

**4. Terraform, file by file ..... 15**
4.1 The lab layer: terraform/ ..... 15
4.2 The hosts map: the inventory of servers ..... 15
4.3 The modules ..... 16
4.4 network-policy.tf: the firewall policy as data ..... 17
4.5 IAM: what the servers are allowed to do in AWS ..... 18
4.6 The persistent and bootstrap layers ..... 18
4.7 Terraform tests ..... 18
4.8 What happens on terraform apply (lab layer) ..... 19

**5. Ansible, playbook by playbook and role by role ..... 20**
5.1 How Ansible runs in this lab ..... 20
5.2 The data files (what you edit day to day) ..... 20
5.3 site.yml: the whole company, in order ..... 21
5.4 The roles ..... 21
5.5 The other playbooks ..... 26

**6. The scripts ..... 27**
6.1 scripts/lab.sh: the lab's front door ..... 27
6.2 scripts/control01-setup.sh ..... 28
6.3 scripts/vpn-client.sh (laptop) ..... 28
6.4 scripts/test-lab.sh: 160 checks ..... 28
6.5 Small helpers ..... 29
6.6 ahmed-backup (util01) ..... 29

**7. How it works end to end ..... 30**
7.1 sara logs into ws-ops01 from home ..... 30
7.2 ahmed (dev) opens a website ..... 30
7.3 A log line travels to util01 ..... 31
7.4 A new host joins the company (enrollment) ..... 31
7.5 A rebuild, start to finish (lab.sh up) ..... 31

**8. Security operations: audit, compliance, patching, drills ..... 32**
8.1 Audit (D-063) ..... 32
8.2 Compliance: CIS Level 1 at 100% (D-064) ..... 32
8.3 Patching cadence (D-065) ..... 33
8.4 Backups and restore drills (D-056, D-066) ..... 34
8.5 Incident drills (D-067) ..... 34

**9. Running the company day to day ..... 35**
9.1 Starting and ending a working session ..... 35
9.2 The change workflow (for every ticket) ..... 35
9.3 Add a user ..... 35
9.4 Change a user ..... 36
9.5 Remove a user (offboarding) ..... 36
9.6 Add a team ..... 37
9.7 Other common tickets ..... 37
9.8 Troubleshooting checklist ..... 37
9.9 Where everything lives (quick reference) ..... 38

**10. Every decision, D-001 to D-069 ..... 39**
10.1 Foundations (D-001 to D-022) ..... 39
10.2 Access, automation and lifecycle (D-023 to D-045) ..... 40
10.3 Identity, VPN policy and util01 (D-046 to D-061) ..... 41
10.4 Roadmap 5 (D-062 to D-069) ..... 42

**11. Incidents and drills ..... 43**
11.1 INC-001: VPN handshakes dropped ..... 43
11.2 INC-002: site.yml froze ..... 43
11.3 INC-003: firewalld "not running" on a fresh host ..... 43
11.4 INC-004: fresh build hung on workstation dnf ..... 43
11.5 INC-005: leaked IAM key used for Amazon Bedrock ..... 43
11.6 INC-006: control01's IP taken by the Instance Connect Endpoint ..... 44
11.7 INC-007: SSH offered password logins on every host ..... 44
11.8 The drills ..... 44

**12. Lessons and traps ..... 45**
12.1 Linux and services ..... 45
12.2 FreeIPA and SSSD ..... 45
12.3 Ansible, SSH and shells ..... 46
12.4 Terraform and AWS ..... 46

**13. Interview preparation ..... 47**
13.1 The 60-second pitch ..... 47
13.2 Design questions ..... 47
13.3 Troubleshooting stories (the best interview material) ..... 48
13.4 Quick-fire technical questions ..... 49
13.5 Known weaknesses (say them before you are asked) ..... 49

**14. Reference ..... 50**
14.1 Command cheat sheet ..... 50
14.2 Repository map ..... 51
14.3 Glossary ..... 51
14.4 What is next ..... 52

# 1. The project in one chapter

## 1.1 What this project is

**ahmed-infra** is a complete, working IT environment for a fictional company called **ahmed**, built on Amazon Web Services (AWS). You play the company's Linux system administrator. The lab contains what a real Linux administrator owns in a company: central user accounts and passwords policy, who may log in where, who may use `sudo`, a VPN for remote staff, firewalls, an internet proxy that decides which websites each team may reach, file servers for home directories and team shares, central logging, security auditing, compliance scanning against an industry benchmark, a patching process, backups with tested restores, and rehearsed incident drills.

Everything is **code**. The servers, networks and firewalls are described in **Terraform**; everything that happens inside the servers is described in **Ansible**. Nothing is configured by hand. To prove that, the whole lab is **destroyed after every working session and rebuilt from Git** at the start of the next one. If something was done by hand and not written into code, it disappears, so it is impossible to cheat.

## 1.2 The company

| Team | Example user | Can log into | sudo | Internet |
|---|---|---|---|---|
| admins | ahmed.admin | every host | everything | standard |
| dev | ahmed | ws-dev01 | none yet (app restart comes with app01) | standard: social media and streaming blocked |
| ops | sara | ws-ops01, idm01, util01 | full | standard |
| dba | omar | ws-ops01 | none yet (`sudo -u postgres` comes with app01) | allow-list only |
| security | laila | util01 | audit tools only: journalctl, ausearch, aureport, last | standard; reads central logs and reports |
| contractors | contractor1 | ws-dev01 | none | allow-list only; account expires 2026-12-31; 50 MB home |

"Standard" internet means any HTTPS site except the deny lists (social media and video/music streaming); AI assistants are allowed because developers use them for work. "Allow-list" means only package repositories, GitHub and documentation. Nobody gets plain HTTP.

## 1.3 The six hosts

| Host | Role | Size | Disks | IP |
|---|---|---|---|---|
| gw01 | VPN gateway, the only host with a public address | t3.small (2 GB) | 20 GiB OS | 10.0.0.10 + Elastic IP |
| control01 | Ansible control node; where every change is applied from | t3.small | 20 GiB OS | 10.0.1.10 |

| | | | | |
|---|---|---|---|---|
| idm01 | FreeIPA: identity, Kerberos, DNS, certificate authority | c7i-flex.large (4 GB) | 20 GiB OS | 10.0.1.11 |
| util01 | Web proxy, NFS file server, central logs and audit, backups | t3.small | 20 GiB OS + 20 GB + 10 GB data | 10.0.1.12 |
| ws-dev01 | Dev team workstation (also contractors) | t3.small | **10 GiB OS only** | 10.0.2.10 |
| ws-ops01 | Ops and DBA workstation | t3.small | **10 GiB OS only** | 10.0.2.11 |

**Why the workstations have no data disks**

Workstations hold no user or team data at all (D-061). Users' home directories and the team shares live on util01's data disks and are mounted over the network (NFS) when a user logs in. A lost or rebuilt workstation therefore loses nothing; the same files appear on any workstation; there is exactly one copy of every file, and it is backed up with util01. This is how company desktops work with network home directories. The workstation disk is 10 GiB because that is the minimum the RHEL image allows.

## 1.4 The principles everything follows

1. **Terraform builds, Ansible configures** (D-003). Terraform creates what AWS provides (network, servers, disks, firewall rules in AWS, IAM permissions). Ansible configures the operating system and the services on top.
2. **Everything is code and the lab is rebuilt every session** (D-032, D-048). Manual repairs are allowed only to rescue a broken host, and must be followed by a code change that prevents the problem from coming back.
3. **Default deny** (D-019). No network connection is possible unless it is written down in `terraform/network-policy.tf`. Firewalls on the hosts repeat the same boundary as a second layer.
4. **Data, not code, for day-to-day changes.** Adding a user, a team, a VPN peer or a website to a block list means editing a data file under `ansible/inventory/group_vars/`, never the logic.
5. **Secrets never in Git, logs or Terraform state** (D-042, D-043). Secrets are encrypted with Ansible Vault; the Vault password lives in AWS SSM and only control01 can read it.
6. **Every decision is recorded** with its alternatives and why it was chosen (`Decisions.md`, D-001 to D-069). Every real failure gets an incident report (INC-001 to INC-007).
7. **Prove it with tests.** `scripts/test-lab.sh` runs 160 checks, logging in as real users and proving both what is allowed and what is refused.

## 1.5 How the project was built (timeline)

| Stage | What was built | Decisions |
|---|---|---|
| Foundations | RHEL 10, region, tool split, network with private subnets and NAT, default-deny security groups, host inventory, S3 state | D-001 to D-022 |
| Access | VPN gateway gw01 (WireGuard), Instance Connect break-glass, the ansible automation account, Ansible from control01 with a dynamic inventory | D-023 to D-031 |
| Lifecycle | `lab.sh`, patch baseline, tests, host keys, VPN fixes (WireGuard inside TLS after INC-001) | D-032 to D-041 |
| Secrets | Persistent Terraform layer, Vault password in SSM, state bucket adopted into code, Terraform tests | D-042 to D-045 |

| | | |
|---|---|---|
| Roadmap 1 | FreeIPA: server, every client enrolled, users and rules as data | D-046 to D-051 |
| Roadmap 2 | Per-team VPN policy, tested with synthetic peers | D-052 to D-055 |
| Roadmap 3 | util01: forced web proxy, NFS homes, central TLS logs, S3 backups | D-056 to D-059 |
| Roadmap 4 | Storage: second disk, quotas, team shares; stable IPA IDs | D-060, D-061 |
| Roadmap 5 | Fixed host IPs reserved; audit; CIS compliance at 100%; patching in rings; restore and incident drills; DNS pinning; admin access | D-062 to D-069 |

## 1.6 How to read this guide

Chapter 2 explains every tool and technology the project uses, from zero, so the rest makes sense. Chapter 3 is the architecture: network, hosts, access paths and the security layers. Chapters 4 and 5 walk through the Terraform and Ansible code file by file and role by role. Chapter 6 explains the scripts. Chapter 7 follows real requests end to end (a login, a VPN packet, a web request, a log line). Chapter 8 covers the security operations (audit, compliance, patching, drills). Chapter 9 is the day-to-day operations manual. Chapter 10 lists every decision, chapter 11 every incident and drill, chapter 12 the lessons and traps, and chapter 13 prepares you for interview questions about the project. Decision numbers (D-0xx), incidents (INC-0xx) and drills (DRILL-00x) refer to `Decisions.md`, `docs/incident-reports/` and `docs/drills/`.

# 2. Tools and technologies, explained from zero

*Each section says what the tool is, what problem it solves, and exactly how this project uses it.*

## 2.1 AWS building blocks

| Service | What it is | How the lab uses it |
|---|---|---|
| **VPC** (Virtual Private Cloud) | Your own private network inside AWS, with an address range you choose. | `10.0.0.0/16`, the company network. |
| **Subnet** | A slice of the VPC's addresses in one Availability Zone. "Public" subnets route to the internet directly; "private" subnets do not. | public `10.0.0.0/24` (gw01, NAT), infra `10.0.1.0/24` (servers), workstation `10.0.2.0/24`, app `10.0.3.0/24` (empty, for app01). |
| **Internet Gateway** | The VPC's door to the internet for public subnets. | Used by gw01 and the NAT gateway. |
| **NAT Gateway** | Lets hosts in private subnets open connections to the internet (for updates) while staying unreachable from it. | Servers download packages through it. About $32/month if left running, which is one reason the lab is destroyed after each session. |
| **Route table** | Decides where packets go next. | Private subnets: default route to the NAT; `10.8.0.0/16` (VPN clients) to gw01 so replies reach VPN users. |
| **Security Group (SG)** | A stateful firewall attached to each server's network interface. Rules say which traffic may enter and leave; replies are allowed automatically. | One SG per host role, default deny, every allowed flow declared (chapter 4.4). |
| **EC2 instance** | A virtual server. | The six hosts. |
| **EBS volume** | A virtual disk attached to an instance. | Encrypted gp3 root disks; util01 has two extra data disks. |
| **Elastic IP** | A public IPv4 address. | gw01 only. It changes on every rebuild, so the VPN client is refreshed after `up`. |
| **IAM role / instance profile** | Permissions given to a server instead of storing passwords or access keys on it. The server gets short-lived credentials automatically. | control01 may list instances and read the Vault password; util01 may read and write its own backups. Nothing else. |
| **IMDSv2** | The instance metadata service (where an instance gets its credentials), in its token-protected version 2. | Required on every host (`http_tokens = "required"`), which blocks a classic credential-stealing attack. |
| **EC2 Instance Connect Endpoint (EICE)** | An AWS-managed tunnel into a private instance, authenticated by AWS credentials, with no public port. | The **break-glass** path to control01 if the VPN is down, and the way `lab.sh` reaches control01. |
| **SSM Parameter Store** | A key-value store for configuration and secrets (SecureString parameters are encrypted with KMS). | Holds the Ansible Vault password. |
| **S3** | Object storage ("buckets"). Versioning keeps old versions of every object. | Terraform state (bucket `ahmed-statefile`) and util01's backups. |

| | | |
|---|---|---|
| **Subnet CIDR reservation** | A block of addresses AWS will never auto-assign. | `10.0.x.8/29` in each subnet, so no AWS-managed interface can take a server's fixed IP (INC-006, D-062). |

## 2.2 RHEL 10 and the operating-system tools

**Red Hat Enterprise Linux 10** is the distribution (D-001): the same platform as the Red Hat certification labs and most enterprises. The official Red Hat image is looked up by name; the hosts are then patched in place to RHEL 10.2.

| Tool | What it does | Used for |
|---|---|---|
| **dnf** | Package manager: installs, updates, downgrades, keeps a history of every transaction (`dnf history`). | Patching, every role's packages, rollback. |
| **systemd** | Starts and supervises services; timers replace cron jobs; `journald` collects logs. | Every service; backup, report and guard timers. |
| **SELinux** | Mandatory access control: every process and file has a label, and policy decides which labels may touch which. Root cannot override it. | Always `enforcing`. Data areas get correct labels (NFS homes labelled like `/home`). |
| **firewalld** | The host firewall (on top of nftables), organised in *zones* and, for routed traffic, *policies*. | Second firewall layer on every host; per-team VPN policy on gw01. |
| **nftables** | The Linux kernel packet filter. | A small extra table on workstations enforces per-user proxy ports. |
| **chrony** | Time synchronisation. | Synchronises with the AWS time service. Kerberos fails if clocks differ by more than 5 minutes. |
| **cloud-init** | Configures an instance on its first boot from "user data". | Sets each host's name and FQDN (`ws-dev01.ahmed.internal`). |
| **OpenSSH** | Remote login. `sshd_config.d/` drop-ins are read in name order, and **the first value read for a keyword wins**. | Key-only logins; our drop-in is `01-ahmed-hardening.conf` so it wins (INC-007). |
| **PAM, authselect** | PAM is the pluggable authentication stack; authselect is RHEL's supported way to configure it through profiles and features. | Custom profile `custom/ahmed` with faillock, password history, no empty passwords. |
| **crypto-policies** | One system-wide setting for allowed ciphers and algorithms used by OpenSSL, OpenSSH, GnuTLS and others. | `DEFAULT:NO-SSHCBC:NO-SSHWEAKCIPHERS:NO-SSHWEAKMACS:NO-WEAKMAC` (CIS). |
| **LVM** | Logical Volume Manager: combines disks into a *volume group* and carves *logical volumes* that can grow online. | util01: volume group `vg_data` over two disks. |
| **XFS** | RHEL's default filesystem; supports user quotas and *project* quotas (a limit for a directory tree). | Home quotas per user, team-share quotas per team. |
| **POSIX ACLs, setgid** | setgid on a directory makes new files inherit its group; a *default ACL* gives new files group write. | Team shares stay writable by the whole team. |

## 2.3 Terraform

Terraform describes infrastructure as code in HCL files. `terraform plan` compares the code with what exists and shows what it would add, change or destroy; `terraform apply` does it. It records what it created in a **state file**, the map between code and real resources. Concepts the lab uses:

- **Remote state with locking:** state is stored in the S3 bucket `ahmed-statefile`, versioned and TLS-only; `use_lockfile = true` creates a lock object in S3 so two runs never write at once (D-016, D-044).
- **Layers:** three separate configurations with separate states: `terraform/bootstrap` (the state bucket itself), `terraform/persistent` (things that survive teardown) and `terraform/` (the lab, destroyed every session) (D-042).
- **Modules:** reusable building blocks with inputs and outputs: `network`, `security-groups`, `ec2-instance` (D-014).
- **`for_each`:** one block creates many resources from a map. Every host comes from one entry in the `hosts` map.
- **Validations:** rules on input variables, for example "two hosts may not share an IP" or "a host's IP must be in its subnet's reserved block".
- **`import` and `moved` blocks:** adopt an existing resource into code without recreating it; rename a resource without destroying it.
- **Ephemeral resources and write-only arguments** (Terraform 1.11+): generate a value and send it to AWS without it ever being stored in state, which is how the Vault password is created.
- **`lifecycle`:** `prevent_destroy` refuses to delete precious resources; `ignore_changes` stops Terraform from replacing servers when a newer image appears; `create_before_destroy` builds a replacement before removing the old one.
- **`terraform test` with mocked providers:** tests that run in seconds without AWS credentials (D-045).

## 2.4 Ansible

Ansible connects to hosts over SSH and runs **modules** (small programs such as "install this package" or "write this file") as described in YAML **playbooks**. It is agentless and **idempotent**: a module changes something only if it is not already in the wanted state, so a second run reports `changed=0`. The lab's terms:

- **Control node:** control01 is the only place Ansible runs, inside a Python virtual environment with pinned versions (D-028, D-030).
- **Dynamic inventory:** the `amazon.aws.aws_ec2` plugin asks AWS which instances exist and groups them by their `Role` and `Team` tags (`role_util`, `team_dev`, ...) (D-029). Nothing can drift because AWS is the source of truth.
- **Roles:** folders of tasks, templates, handlers and defaults for one job (`base`, `nfs_server`, ...).
- **group_vars:** data per group, the files you edit day to day.
- **Handlers:** tasks that run only when something changed, for example "restart sshd after its config changed".
- **Templates (Jinja2):** config files with variables, rendered per host.
- **Collections:** packaged modules and roles: `ansible.posix`, `amazon.aws`, `freeipa.ansible_freeipa`, pinned in `requirements.yml`.

- **Ansible Vault:** encrypts whole files of secrets. The password comes from SSM through `scripts/vault-password.py`.
- **become:** run as root through sudo. **pipelining** streams modules over SSH without temporary files.

## 2.5 FreeIPA (Red Hat Identity Management)

FreeIPA is Red Hat's central identity system, the Linux equivalent of Microsoft Active Directory (D-006). One server (idm01) bundles several well-known components:

| Component | Job |
|---|---|
| **389 Directory Server (LDAP)** | The database of users, groups, hosts, rules and SSH keys. |
| **MIT Kerberos KDC** | Single sign-on tickets; every enrolled host gets a *keytab*, its own Kerberos identity. |
| **BIND DNS** (integrated) | Serves `ahmed.internal`: host records created automatically at enrollment, service records, SSHFP records (SSH host key fingerprints), reverse (PTR) records. |
| **Dogtag CA** | A certificate authority that issues TLS certificates to hosts and services. The lab uses it for rsyslog's TLS. |
| **certmonger** | On clients: requests certificates from the CA and renews them before they expire. |
| **HBAC rules** | Host-Based Access Control: *which users may log into which hosts*. The lab has no default "allow_all" rule. |
| **sudo rules** | Central sudo policy: who may run what as whom on which hosts. |
| **Automount maps** | Central autofs maps: `/users/<name>` and `/teams/<team>` come from util01 over NFS. |
| **Web UI and API** | Management over HTTPS (also from the admin VPN range). |

On every other host, **SSSD** (System Security Services Daemon) is the client side: it answers "who is `sara`?" (UID, groups, home), fetches her SSH public key for sshd, checks HBAC rules during login, delivers sudo rules, serves automount maps, and **caches** all of it so known users keep working when idm01 is briefly down. A host joins IPA by **enrollment** (`ipa-client-install`), using a one-time password that the playbook creates for it.

## 2.6 Networking and remote access tools

| Tool | What it is | In this lab |
|---|---|---|
| **WireGuard** | A modern, minimal VPN in the Linux kernel. Each peer has a key pair; a server accepts only packets signed by a known peer key, so an unknown sender gets no answer at all. | The company VPN on gw01; one key and one fixed VPN address per user. |
| **wstunnel** | Carries UDP or TCP traffic inside a TLS WebSocket connection on TCP 443, so it looks like HTTPS. | WireGuard's UDP was dropped on Ahmed's network (INC-001), so WireGuard travels inside wstunnel on TCP 443 (D-040). The pinned binary is checksum-verified. |
| **Squid** | A web proxy. With HTTPS it sees only the destination domain (the CONNECT request), never the content, unless it intercepts TLS, which this lab does not. | On util01: forced proxy for workstations, one port per policy (3128 standard, 3129 allow-list). |

| | | |
|---|---|---|
| **NFSv4** | Network File System: a server exports directories that clients mount. Version 4 needs only TCP 2049. | util01 exports homes and team shares to the two workstations only. |
| **autofs** | Mounts network directories on demand when someone opens them, and unmounts them when idle. | Workstations mount `/users/<name>` at login and `/teams/<team>` when used. |

## 2.7 Logging, audit and security tools

| Tool | What it is | In this lab |
|---|---|---|
| **rsyslog** | The classic syslog daemon; can send and receive logs over TCP with TLS, and queue messages on disk. | Every host forwards everything to util01 on TCP 6514, TLS with IPA certificates; util01 stores `/srv/logs/<fqdn>/<program>.log`. |
| **auditd** | The Linux audit system: the kernel records security events (file changes, commands, logins) according to rules; `ausearch` searches the records, `aureport` summarises them. | Keyed rules on every host; records forwarded to util01 by the audit syslog plugin. |
| **OpenSCAP** and **SCAP Security Guide** | OpenSCAP is a scanner; the SCAP Security Guide (SSG) ships benchmark profiles such as CIS and DISA STIG. Terms: *XCCDF* (the checklist and results format), *OVAL* (how each check is tested), *profile* (a selection of rules), *tailoring file* (your own adjustments to a profile). | CIS Level 1 scans on every host; results on util01 for the security team; 100% with documented exceptions. |
| **CIS Benchmarks** | Security configuration baselines by the Center for Internet Security. Level 1 is practical and meant not to break services; Level 2 is stricter. | CIS Level 1 Server for servers, Workstation for workstations (D-064). |
| **AIDE** | Advanced Intrusion Detection Environment: records checksums of system files in a database and reports any change. | Database rebuilt after every Ansible run and patch; nightly check reports changes made outside Ansible. |
| **faillock, pwquality, pwhistory** | PAM modules: lock an account after failed passwords; enforce password strength; remember old passwords. | 5 failures lock 15 minutes; 14-character, 4-class passwords; last 24 remembered. |

## 2.8 Backups and the AWS CLI

Backups use two standard tools: **tar** (archives files with their permissions, ACLs, extended attributes and SELinux labels, and stores owners *by name*) and **awscli2** (streams the archive straight into S3 with no temporary file). The bucket is versioned, so every backup keeps the previous ones for 30 days. No third-party backup software or repository is needed (D-056).

## 2.9 Where each tool came from

Everything comes from Red Hat's own repositories except: the Ansible collections (Ansible Galaxy, pinned versions), `wireguard-tools` (one package from EPEL through a repository definition limited to that package, D-033), and the wstunnel binary (GitHub release, pinned version, SHA-256 verified). Keeping third-party sources to these three is deliberate.

# 3. Architecture

## 3.1 The network

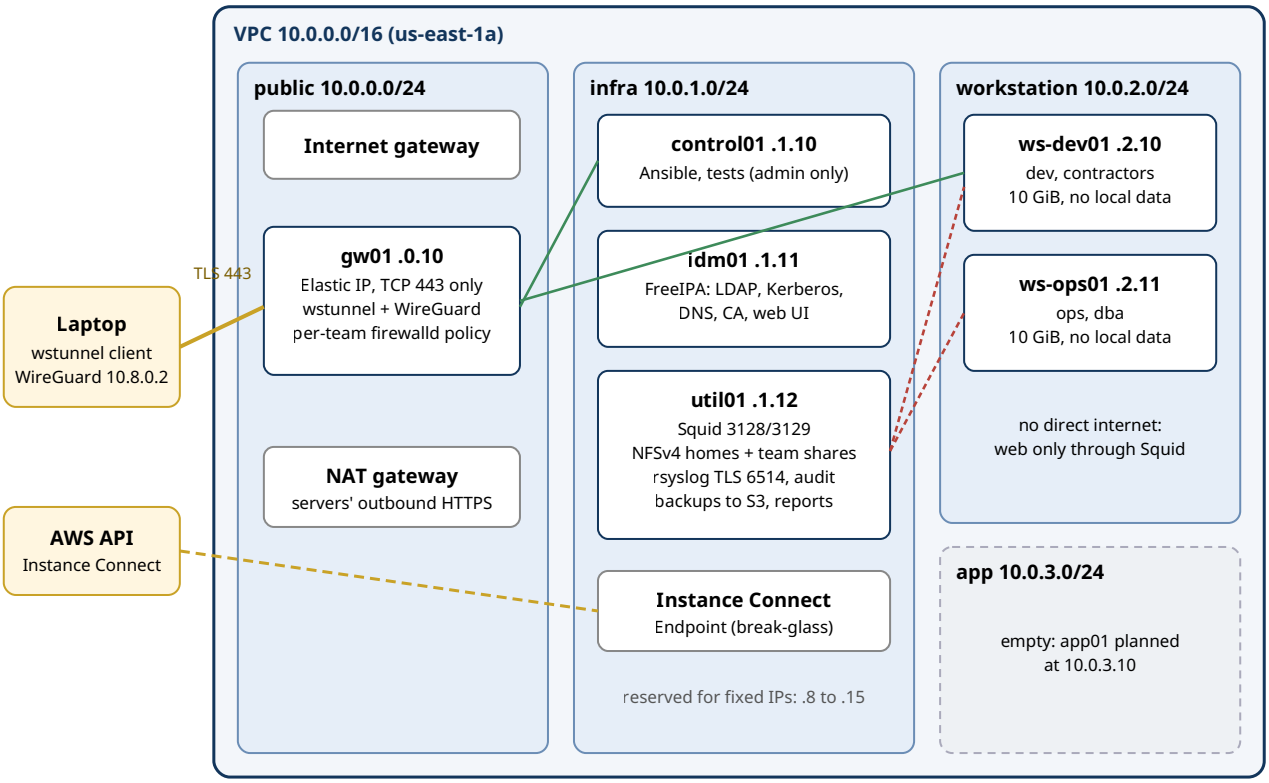



Figure 1. The company network. Green: forwarded VPN traffic. Red dashed: NFS and proxy traffic between the workstations and util01. Yellow: the only two ways in (VPN on TCP 443, and Instance Connect through the AWS API).

| Subnet | CIDR | Type | Reserved for fixed IPs | Hosts |
|---|---|---|---|---|
| public | 10.0.0.0/24 | route to the internet gateway | 10.0.0.8/29 | gw01 (10.0.0.10), NAT gateway |
| infra | 10.0.1.0/24 | private, default route to NAT | 10.0.1.8/29 | control01, idm01, util01, Instance Connect Endpoint |
| workstation | 10.0.2.0/24 | private, but no internet egress in the security groups | 10.0.2.8/29 | ws-dev01, ws-ops01 |
| app | 10.0.3.0/24 | private | 10.0.3.8/29 | empty (app01) |

VPN clients use `10.8.0.0/16`, split per team: admin `10.8.0.0/24`, dev `10.8.1.0/24`, ops `10.8.2.0/24`. VPN traffic is **routed, not NATed** (D-024): when sara connects, servers see her own VPN address (10.8.2.x), so firewalls can tell teams apart and logs show who it was. That requires two things: gw01's "source/destination check" is disabled so AWS lets it forward packets, and the private route table sends `10.8.0.0/16` back to gw01.

## 3.2 Who reaches what

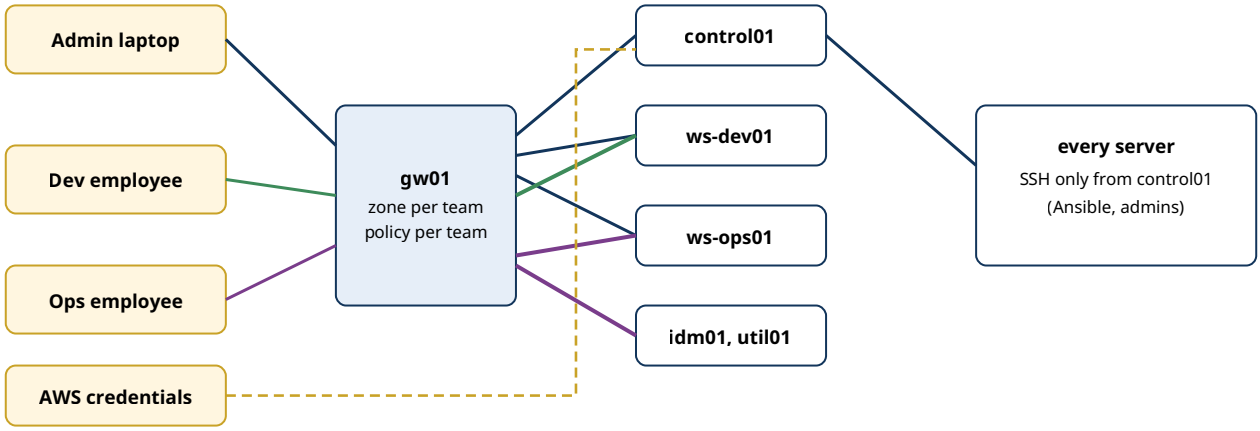



Figure 2. Access paths. Blue: admin range (control01, both workstations, IPA web UI, SSH to gw01). Green: dev range (ws-dev01 only). Purple: ops range (ws-ops01, idm01, util01). Dashed: break-glass through AWS Instance Connect.

- **Admin:** laptop, then the VPN (admin range), then `ssh control01-vpn`, then any host. If the VPN is broken: `ssh control01` through Instance Connect, authenticated by AWS credentials.
- **Employees:** laptop, then the VPN (their team's range), then their team's workstation only, which simulates sitting at their own desk. Ops may also reach idm01 and util01 (D-053).
- **Servers** accept SSH only from control01, plus the per-team VPN exceptions above.
- **The only port open to the internet** is TCP 443 on gw01, which carries WireGuard inside TLS. WireGuard ignores any packet not signed by a known peer key, so the open port reveals nothing.

## 3.3 Defence in depth: the layers a connection must pass

| # | Layer | Where defined | What it decides |
|---|---|---|---|
| 1 | WireGuard peer keys | `role_gateway/main.yml` | Only known devices can talk to the VPN at all |
| 2 | gw01 firewalld zones and policies | `vpn_team_policy` in the same file | Which hosts and ports each team's VPN range may reach |
| 3 | AWS security groups | `terraform/network-policy.tf` | Which role may talk to which role or IP range, on which port |
| 4 | Host firewalld | Ansible roles | The same boundary again on each host |
| 5 | sshd: keys only | `base` role, `01-ahmed-hardening.conf` | No passwords, no root, public keys only (tested) |
| 6 | IPA HBAC rules | `role_idm/identity.yml` | Which users may log into which hosts |
| 7 | IPA sudo rules | same file | What each user may run as root |
| 8 | SELinux, file permissions, quotas | roles | What processes and users may touch once logged in |
| 9 | Audit, AIDE, central logs | `audit`, `hardening`, log roles | Every privileged action and change is recorded off the host |

Two independent layers enforce each network boundary (layers 2 and 3, then 3 and 4), so one mistake does not open a path (D-052).

## 3.4 Where the data lives (and why)

| Data | Lives on | Survives a rebuild because |
|---|---|---|
| Code, configuration, users, rules | Git (GitHub) | it is the source of truth; everything is rebuilt from it |
| Secrets (IPA passwords, gw01 WireGuard key, test keys) | Ansible Vault files in Git, encrypted | the encrypted files are in Git; the Vault password is in SSM (persistent layer) |
| Ansible Vault password | SSM Parameter Store | persistent Terraform layer, `prevent_destroy` |
| Terraform state | S3 bucket `ahmed-statefile` | bootstrap layer, versioned, never destroyed |
| User homes, team shares, central logs, reports | util01 `vg_data`, backed up to S3 | backed up before `down`, restored onto the new volume on `up` |
| Local state on servers (packages, caches) | each host's root disk | it does not: rebuilt by Ansible |

util01's two data disks (20 GB and 10 GB) form the volume group `vg_data`, with three logical volumes:

| Logical volume | Size | Mounted at | Quota | Seen on the workstations as |
|---|---|---|---|---|
| lv_home | 12 GB | /export/home | XFS user quota: default soft 1 GB / hard 1.5 GB; contractor1 50 MB | /users/<name> (autofs) |
| lv_teams | 8 GB | /export/teams | XFS project quota per team: 2 GB (contractors 1 GB, company 4 GB) | /teams/<team> (autofs) |
| lv_logs | 6 GB | /srv/logs | none; a space guard keeps it under 85% | (util01 only) |

About 4 GB of the volume group is left free for growth. Volumes grow online with `lvextend --resizefs`, and a disk is added by adding an EBS volume in Terraform (D-061).

## 3.5 The three Terraform layers

| Layer | Directory | State key | Contents | Lifetime |
|---|---|---|---|---|
| bootstrap | `terraform/bootstrap` | ahmed-infra/bootstrap.tfstate | The state bucket itself (adopted with import) | permanent, `prevent_destroy` |
| persistent | `terraform/persistent` | ahmed-infra/persistent.tfstate | SSM Vault password; S3 backup bucket | permanent, `prevent_destroy` |
| lab | `terraform/` | ahmed-infra/terraform.tfstate | VPC, subnets, NAT, security groups, hosts, disks, IAM roles, Instance Connect, VPN route (169 resources) | destroyed after every session |

A separate state per layer makes the boundary structural: `terraform destroy` in the lab layer simply cannot see the persistent resources (D-042).

# 4. Terraform, file by file

*Terraform creates everything AWS provides. This chapter explains every file, so you can read any line of the code and know why it is there.*

## 4.1 The lab layer: `terraform/`

| File | What it contains |
|---|---|
| `versions.tf` | Terraform `~> 1.11`, AWS provider `~> 6.0` (locked at 6.70.0 in `.terraform.lock.hcl`), and the S3 backend: bucket `ahmed-statefile`, key `ahmed-infra/terraform.tfstate`, encryption, `use_lockfile = true`. |
| `providers.tf` | The AWS provider with **default tags** on every resource: `Company=ahmed`, `Project=ahmed-infra`, `ManagedBy=terraform`. The Ansible inventory filters on `Project`. |
| `variables.tf` | Every input: region, company name, domain `ahmed.internal`, the VPC and subnets (with each subnet's reserved `static_cidr`), the VPN ranges, the image name pattern, and the **`hosts` map**, the inventory of every server. Validations reject bad input before anything is built. |
| `terraform.tfvars` | Local settings (gitignored; copy from `terraform.tfvars.example`). Most importantly `enable_vpn_endpoint = true`, which opens TCP 443 on gw01. `lab.sh` refuses to run without this file. |
| `main.tf` | Calls the `network` and `security-groups` modules. |
| `network-policy.tf` | The whole network policy as data: roles, flows between roles, and IP-range rules (section 4.4). |
| `hosts.tf` | One `ec2-instance` module call with `for_each = var.hosts`: every server comes from one map entry. |
| `compute.tf` | Looks up the official Red Hat RHEL 10 image by name and owner (D-021), and imports the public half of the bootstrap SSH key (the private key never enters Terraform, D-004). |
| `iam.tf` | Two instance roles (section 4.5). |
| `access.tf` | The EC2 Instance Connect Endpoint in the infra subnet: admin entry for `lab.sh` and break-glass (D-026). |
| `vpn.tf` | The route that sends replies for `10.8.0.0/16` from the private subnets to gw01's network interface (D-024). |
| `moved.tf` | `moved` blocks from a 2026-11-01 rename of the control01 role, so the state kept its history instead of destroying and recreating it. |
| `outputs.tf` | VPC and subnet IDs, security group IDs, gw01's public IP, the VPN endpoint URL, each host's private IP and instance ID. |
| `tests/hosts.tftest.hcl` | Mocked tests of the host validations (section 4.7). |

## 4.2 The `hosts` map: the inventory of servers

Every host is one entry. The fields:

| Field | Meaning | Example |
|---|---|---|
| `role` | Its job; becomes the `Role` tag, so the Ansible group `role_<role>`, and its default security group | `util` |

| | | |
|---|---|---|
| `subnet` | Which subnet | `infra` |
| `private_ip` | Fixed address, which must lie in the subnet's reserved block `10.0.x.8/29` | `10.0.1.12` |
| `instance_type` | Size | `t3.small`; idm01 `c7i-flex.large` (4 GB, free-plan eligible, D-025) |
| `team` | `Team` tag, so the group `team_<team>` | `platform`, `dev`, `ops` |
| `allocate_eip` | Public Elastic IP | only gw01 |
| `source_dest_check` | `false` lets the host forward traffic that is not its own | only gw01 (VPN router) |
| `control_node_role` | Gets the control-node IAM role | only control01 (a validation allows exactly one) |
| `backup_writer` | Gets the backup IAM role | only util01 |
| `security_group` | Overrides the role's group | workstations: `workstation-dev` / `workstation-ops` |
| `root_volume_size` | OS disk in GiB (default 20) | workstations: 10 (no data on them, D-061) |
| `extra_volumes` | Extra EBS disks | util01: `data` 20 GB on `/dev/sdf`, `data2` 10 GB on `/dev/sdg` |

Validations on the map: every subnet exists; every role or security group is a known role; every IP is a valid IPv4 address; no two hosts share an IP; every IP lies inside its subnet's reserved block (D-062); root volumes are at least 10 GiB; exactly one control node. Adding a server is therefore one entry, and Terraform catches mistakes before AWS sees them.

## 4.3 The modules

**`modules/network`**

- The VPC (DNS support and hostnames on), an internet gateway, one subnet per entry of `var.subnets` (public subnets give instances public IPs; private ones never do).
- **Subnet CIDR reservations** (`aws_ec2_subnet_cidr_reservation`, type `explicit`) for `10.0.x.8/29` in every subnet. AWS never hands those addresses to managed interfaces such as the NAT gateway or the Instance Connect Endpoint (INC-006, D-062). The module's `subnet_ids` output and the NAT gateway `depends_on` the reservations, so nothing is placed in a subnet before its addresses are protected.
- A NAT gateway with its Elastic IP in the public subnet, switchable with `enable_nat_gateway`.
- Route tables: public (default route to the internet gateway) and private (default route to the NAT).
- Mocked tests (`tests/nat.tftest.hcl`): NAT on by default, NAT off removes all NAT cost, only public subnets map public IPs, every subnet has an explicit reservation.

**`modules/security-groups`**

- One empty security group per role. "Empty" means everything is denied.
- The VPC's built-in default security group is taken over and stripped of all rules, so nothing can accidentally use it.
- For every **flow** (role A to role B on a port) it creates **both halves**: an egress rule on A that references B's group, and an ingress rule on B that references A's group. With default-deny egress, a connection needs both.

- `cidr_ingress` and `cidr_egress` maps create rules for IP ranges (the VPN ranges, the VPC, the internet).
- Rules are separate resources (`aws_vpc_security_group_*_rule`), never inline, which avoids circular references between groups (D-015).
- Mocked tests: one group per role, each flow opens exactly its two halves and nothing else, and flows or rules naming an unknown role are rejected.

#### `modules/ec2-instance`

- The instance, with its fixed private IP, security group, bootstrap key and optional IAM instance profile.
- **cloud-init user data** sets the hostname and FQDN (`<name>.ahmed.internal`), which FreeIPA requires.
- **IMDSv2 required**, hop limit 1: containers or proxies on the host cannot reach the metadata service.
- An encrypted gp3 root disk; extra encrypted gp3 volumes and their attachments from `extra_volumes`.
- An optional Elastic IP.
- `lifecycle { ignore_changes = [ami, user_data] }`: when Red Hat publishes a newer image, Terraform does **not** replace the servers. They are patched in place with dnf instead (D-021, D-034).

## 4.4 `network-policy.tf`: the firewall policy as data

The file defines four things, and the security-groups module turns them into rules:

1. **Roles:** `eice, gateway, control, idm, util, workstation-dev, workstation-ops`. Each team's workstation has its own group, so AWS enforces the same per-team boundary as gw01's firewall (D-052).
2. **Flows** (role to role):
   - Instance Connect to control01 on SSH; control01 to every other role on SSH (Ansible).
   - Workstations to util01 on 3128 and 3129 (Squid), on 2049 (NFSv4).
   - Every role except util to util01 on 6514 (logs over TLS).
3. **Ingress from IP ranges**:
   - The internet to gw01 on TCP 443, only if `enable_vpn_endpoint`.
   - The admin VPN range to control01 and both workstations on SSH and ICMP.
   - Each team's VPN range to its own workstation on SSH; the ops range to idm01 and util01 on SSH (D-053).
   - The whole VPC to idm01 on the FreeIPA ports (DNS 53 TCP/UDP, Kerberos 88 and 464 TCP/UDP, LDAP 389, LDAPS 636, HTTP 80 for certificate revocation, HTTPS 443); the admin range to idm01 on 443 (web UI); ICMP inside the VPC.
4. **Egress to IP ranges**:
   - Servers (not workstations) may use HTTPS to the internet through the NAT, for packages.
   - Every role may send ICMP inside the VPC.
   - Every role may reach idm01's single address on the FreeIPA ports (HTTPS only for workstations, since servers already have it).
   - gw01 may forward VPN SSH into the infra and workstation subnets: security group egress also applies to forwarded packets (D-024).

**Why FreeIPA uses the VPC range instead of role-to-role flows**

Every host in the VPC is an IPA client by design, so "from the VPC" is the honest boundary, and each host's egress is still pinned to idm01's single /32. Role-to-role flows for ten ports and six roles would be about 72 rules, near AWS's default limit of 60 rules per group (D-051).

**Opening a new connection (the change procedure)**

Add one entry to `flows` (or the CIDR maps), run `terraform plan`, check that it adds exactly the two rules you expect, apply, and record it. Anything not written here stays denied.

## 4.5 IAM: what the servers are allowed to do in AWS

| Role | Given to | Permissions | Why so narrow |
|---|---|---|---|
| `ahmed-control-node` | control01 | `ec2:DescribeInstances`, `DescribeRegions`, `DescribeAvailabilityZones` (dynamic inventory); `ssm:GetParameter` on `/ahmed-infra/*` only; `kms:Decrypt` only when SSM makes the call | No stored credentials; cannot write anything; cannot read other secrets. Policies use `create_before_destroy` so a rename never leaves control01 without permissions. |
| `ahmed-backup-writer` | util01 | List the backup bucket; get and put objects under `util01/*` | **No delete.** With versioning, an overwrite keeps the old version, so a compromised util01 cannot erase the backup history. |

## 4.6 The persistent and bootstrap layers

`terraform/persistent`

- **The Ansible Vault password**: an `ephemeral "random_password"` (64 characters) written to the SSM SecureString `/ahmed-infra/ansible-vault-password` through the **write-only** argument `value_wo`. The password never appears in plan output, in state or on disk (D-043). Rotation means re-keying every vault file and bumping `vault_password_version`.
- **The backup bucket** `ahmed-infra-backups-<account>`: versioning (old versions expire after 30 days), public access blocked, SSE-S3 encryption, owner-enforced objects (no ACLs), and a policy that denies any request not made over TLS. `prevent_destroy` on everything.

`terraform/bootstrap`

The state bucket `ahmed-statefile` existed before the code did, so it is **adopted with `import` blocks** rather than recreated (D-044): versioning (old state versions expire after 90 days, so an overwritten state can be recovered), public access blocked, encryption, a TLS-only policy, `prevent_destroy`. Its own state lives in the same bucket. If the bucket were ever lost, the backend block is commented out and the layer applied locally.

## 4.7 Terraform tests

`terraform test` runs plans against a **mocked AWS provider**, so no credentials are needed, nothing is created, and they take seconds (D-045). The root tests check that the default six-host inventory plans, and that each validation rejects what it should: an unknown subnet, an unknown role, a duplicate IP, two control nodes, an IP outside the reserved block. The module tests are listed in 4.3. Run them with:

```
terraform -chdir=terraform test
terraform -chdir=terraform/modules/network init -backend=false && terraform -chdir=terraform/
modules/network test
terraform -chdir=terraform/modules/security-groups init -backend=false && terraform -
chdir=terraform/modules/security-groups test
```

## 4.8 What happens on `terraform apply` (lab layer)

1. VPC, internet gateway, subnets, route tables, the default security group stripped.
2. Subnet CIDR reservations; only then the NAT gateway (and its EIP), the subnet outputs and anything placed in a subnet.
3. Security groups and their rules (about 120 rule resources).
4. Key pair, IAM roles and instance profiles.
5. The six instances (with fixed IPs inside the reservations), util01's data volumes and attachments, gw01's EIP.
6. The Instance Connect Endpoint and the VPN return route.

The whole layer is 169 resources. `terraform plan -destroy` before every teardown confirms that only those are touched: never the SSM parameter, the backup bucket or the state bucket.

# 5. Ansible, playbook by playbook and role by role

## 5.1 How Ansible runs in this lab

- **Only on control01**, from `~/ahmed-infra/ansible`, inside the virtual environment `~/ansible-venv` (ansible-core 2.18, boto3). Changes are made on the laptop, committed and **pushed**; control01 pulls them. Git is the only way code reaches the servers (D-028).
- It connects as the local **`ansible`** account on every host, with the key `~/.ssh/ansible_ed25519` generated on control01, and passwordless sudo. The account is local, not an IPA user, so automation works even when IPA is down, and while IPA is being installed (D-027).
- **`ansible.cfg`**: inventory `inventory/aws_ec2.yml`; 10 forks; every run logged to `~/ansible.log`; any task over 30 minutes fails instead of hanging; the vault password from `scripts/vault-password.py`; become with sudo; pipelining; SSH keepalives so a dead connection is noticed within a minute.
- **Dynamic inventory** (`aws_ec2.yml`): running instances tagged `Project=ahmed-infra`, named by their `Name` tag, reached on their private IP (control01 itself through a local connection). The SSH host key is remembered under the **instance ID** (`HostKeyAlias`), so a replaced server at the same IP is recognised as a new machine instead of triggering a false alarm (D-036). Groups: `role_*` and `team_*` from the tags, `infra`, and the groups the FreeIPA collection expects: `ipaservers` (idm01) and `ipaclients` (everything else).
- Pinned collections (`requirements.yml`): `ansible.posix` 2.2.2, `amazon.aws` 12.0.0, `freeipa.ansible_freeipa` 1.17.0. They are bumped on purpose, in their own commit (D-030).

## 5.2 The data files (what you edit day to day)

| File | Contents |
|---|---|
| `group_vars/all.yml` | The `ansible` user and key; the IPA domain and realm; the proxy host and ports; which teams get the allow-list policy; the compliance minimum scores and every documented CIS exception, with its reason |
| `group_vars/ipaclients.yml` | Enrollment settings: full server names, idm01 as the first DNS server, SSHFP publishing |
| `group_vars/role_control.yml` | control01 runs modules with the system Python (to see IPA's Python libraries) |
| `group_vars/role_idm/main.yml` | The IPA server install: realm, integrated DNS with forwarding to the VPC resolver, reverse zones per subnet, no `allow_all` HBAC rule, the fixed ID range 455400000 to 455599999 |
| `group_vars/role_idm/identity.yml` | **The company's people**: users (fixed UID, groups, SSH keys, quota, expiry), groups (fixed GID), host groups, HBAC rules, sudo rules, automount settings. The next free ID is noted in the file. |
| `group_vars/role_idm/vault.yml` | Encrypted: the IPA admin and Directory Manager passwords |
| `group_vars/role_gateway/main.yml` | VPN peers (name, team, fixed VPN address, public key), the server's public key, and `vpn_team_policy` (what each team's range may reach) |
| `group_vars/role_gateway/vault.yml` | Encrypted: gw01's WireGuard private key, so the server key survives rebuilds |

| | |
|---|---|
| `group_vars/role_util/storage.yml` | Default home quota, team shares and their quotas, the company share |
| `group_vars/role_util/web_policy.yml` | Squid's deny lists (social, streaming) and the allow-list |
| `tests/*.vault.yml` | Encrypted private test keys for the IPA users and the VPN test peers (used only by `test-lab.sh`) |

## 5.3 `site.yml`: the whole company, in order

One playbook configures everything. The order matters, and each step exists for a reason:

| # | Play | Hosts | Why here |
|---|---|---|---|
| 1 | Patch util01 first (`patch.yml`) | util01 | Workstations reach package repositories only through Squid on util01, so util01 must be patched and serving first (INC-004) |
| 2 | `base`, `web_proxy` | util01 | Squid exists before any workstation needs packages |
| 3 | dnf through the proxy (`proxy_client` tasks_from dnf) | workstations | Set by IP address: names do not resolve before IPA exists |
| 4 | Patch everyone else | all except util01 | One patch level for the whole fleet before configuring (D-034) |
| 5 | `base` | all | Baseline: packages, time, SELinux, firewall, SSH, banners, pinned names |
| 6 | FreeIPA (`idm.yml`) | idm01, then clients | Identity before anything that needs users, certificates or DNS |
| 7 | `util_storage`, `log_server`, `nfs_server`, `team_shares` | util01 | The data volume first (homes and logs live on it); its owners resolve through IPA |
| 8 | `log_forwarder` | all | Needs IPA certificates and util01's log server |
| 9 | `audit` | all | Its records travel through the forwarder |
| 10 | `nfs_client`, `proxy_client` | workstations | Need IPA's automount maps and user IDs |
| 11 | `wireguard_gateway` | gw01 | The VPN |
| 12 | WireGuard tools for the test peers | control01 | So `test-lab.sh` can act as a VPN client of each team |
| 13 | `hardening` | all | **Last**: every service is installed with stock settings first, then the host is hardened, then the tests prove everything still works (D-064) |

A second run of `site.yml` with no code changes reports **`changed=0` on every host**. That idempotency is checked by `lab.sh verify --full`.

## 5.4 The roles

### base (every host)

- Installs troubleshooting packages: vim, tmux, rsync, lsof, tcpdump, bind-utils, ncat, traceroute, SELinux tools, firewalld, chrony.
- **Pins idm01 and util01 in `/etc/hosts`** from the inventory's fixed IPs, so the proxy, NFS and log server stay reachable even if IPA's DNS is down (D-068, DRILL-001).

- UTC timezone and chrony; SELinux enforcing; firewalld running. It waits for `firewall-cmd --state`, because systemd reports firewalld "started" before it is ready (INC-003). SSH is allowed in the host firewall.
- The SSH hardening drop-in **`/etc/ssh/sshd_config.d/01-ahmed-hardening.conf`**, mode 0600. It is named `01-` on purpose: sshd keeps the **first** value it reads, and IPA's `04-ipa.conf` had been turning keyboard-interactive (PAM password) logins back on (INC-007). Settings:
  - Public keys only: no passwords, no keyboard-interactive, no GSSAPI, no host-based login, no root login, no user environment.
  - X11 forwarding off; on workstations all forwarding is off.
  - MaxAuthTries 4, MaxSessions 10, MaxStartups 10:30:60, LoginGraceTime 30.
  - A silent client is dropped after one unanswered 300 s check; LogLevel VERBOSE; a banner.

  The whole configuration is validated with `sshd -t` before any restart, so a typo cannot lock everyone out.
- Banners: `/etc/issue.net` and `/etc/issue` carry the company warning, with no OS details; a post-login message.

### FreeIPA: `idm.yml` and five roles

1. **Server** (`freeipa.ansible_freeipa.ipaserver` on idm01, `state: present` passed explicitly), then **`ipa_server_verify`**: every IPA service runs; admin gets a Kerberos ticket and can use the API; DNS answers for `ahmed.internal` and forwards other names.
2. **Clients**, two at a time, control01 last and alone (it runs the playbook). Before each enrollment, **`ipa_client_prepare`**:
   - removes an empty `/etc/ipa/ca.crt` left by a failed attempt;
   - creates a **one-time enrollment password** on idm01, so the client never sees the admin password;
   - releases the IPA entry of a host that was rebuilt, because IPA refuses a one-time password for a host it still considers enrolled;
   - registers the host with its DNS record.

   Then `ipaclient` enrolls it, and **`ipa_client_health`** checks that SSSD is online and resolves IPA users, restarting an SSSD stuck offline.
3. **`ipa_identity`** on idm01: groups, users (SSH keys, home `/users/<name>`, expiry), memberships, the company group `employees`, host groups, HBAC rules, PTR records for every host (rsyslog names log directories by reverse DNS), the automount maps for homes and team shares, sudo commands and rules. Everything comes from `identity.yml`.
4. **`ipa_client_config`** on every client: labels and creates `/users`, records a checksum of the identity data, and **resets SSSD's cache when it changes**. SSSD otherwise keeps an empty sudo-rule cache from before the rules existed.

> **The `state` trap**
> The AWS inventory gives every host a variable `state` (the instance state, "running"). The FreeIPA roles use a variable of the same name to choose install or *uninstall*, so both were silently skipped. Passing `state: present` as a role parameter outranks the inventory variable.

### util_storage (util01)

- Installs LVM and the AWS CLI. It finds the data disks (the unpartitioned NVMe disks) and refuses to continue if their number differs from Terraform's.
- Adds only **blank** disks to `vg_data`: a disk with any filesystem or LVM signature stops the run instead of being overwritten.
- Creates or grows `lv_home`, `lv_teams` and `lv_logs` (online `lvextend --resizefs`), makes XFS filesystems and mounts them permanently. Quota accounting (uquota, prjquota) is switched on at mount time, because XFS cannot enable it with a remount.
- SELinux labels: homes are labelled like `/home` (equivalence rule), logs as `var_log_t`, team shares as `public_content_rw_t`.
- Installs `/usr/local/sbin/ahmed-backup` (backup, restore, verify, drill) and a daily timer at 02:00.
- **Restores the latest backup only onto a volume group created in this run**, so it never overwrites live data.

### log_server (util01)

- rsyslog with the OpenSSL TLS driver listens on TCP 6514 and accepts only senders presenting a valid IPA certificate.
- It stores `/srv/logs/<sender FQDN>/<program>.log`, with the host name taken from the sender's reverse DNS rather than from the message, so one host cannot write into another's directory.
- Audit records go raw into `audit.log`, so `ausearch -if` reads them.
- Directory and file modes 0750/0640 with group `security` (laila). The service `UMask` is overridden to 0027, because rsyslog's default turned 0640 into 0600.
- 14 days of rotation.
- The daily audit report: the script `ahmed-audit-report` plus an hourly timer.
- The **log space guard** `ahmed-logspace` runs every 5 minutes. Above 85% use it deletes the oldest rotated logs until use is under 80%, because rsyslog drops messages it cannot write (DRILL-002).
- The log port is open in the host firewall to the VPC only.

### log_forwarder (every host)

certmonger requests the host's own certificate from the IPA CA (`ipa-getcert`) and renews it automatically. rsyslog then forwards every message to `util01.ahmed.internal:6514` over TLS, checking util01's name. A disk-assisted queue (200 MB, infinite retry) keeps messages while util01 is unreachable.

### audit (every host)

Installs auditd and its syslog plugin (`audispd-plugins`). Every record is tagged with the host name (`name_format = HOSTNAME`), and audit lines are kept out of the local `/var/log/messages`. The rules are in `/etc/audit/rules.d/50-ahmed.rules`, each with a key:

| Key | Watches |
|---|---|
| identity | writes to passwd, group, shadow, gshadow, opasswd |
| sudoers | /etc/sudoers and sudoers.d |
| sshd | the SSH server configuration |

| | |
|---|---|
| identity-config | SSSD, krb5 and /etc/ipa (how the host trusts IPA) |
| time-change | settimeofday, clock_settime, /etc/localtime |
| modules | kernel module load and unload |
| audit-config | changes to the audit configuration itself |
| privileged | every command a real user (UID 1000 and up) runs as root, including inside `sudo -i`; the ansible account is excluded (its changes are traced in Git and ansible.log) |

The role checks that every keyed rule loaded, because `augenrules` skips a bad line silently (D-063).

### nfs_server (util01)

- NFSv4 over TCP only (v3 and UDP off).
- Exports `/export/home` and `/export/teams` to the two workstation addresses only, `sec=sys`, `root_squash`; NFS is open in the host firewall to the workstations only.
- Creates every IPA user's home from `/etc/skel` (copying its contents, not `/etc/skel/.`, whose 0755 mode once opened every home), re-owns files by name after a restore, keeps homes private and sets each user's quota.
- Applies SELinux labels and confirms that NFSv3 is really off and that the exports are active.

### team_shares (util01)

Creates `/export/teams/<team>` mode 2770, so setgid gives new files the team's group and nobody else may enter. A **default ACL** keeps new files group-writable whatever the creator's umask. `company` is mode 3770 with group `employees`: the sticky bit lets only a file's owner delete it. Each share gets an XFS project ID equal to the group's fixed GID (stable across rebuilds) and its quota.

### nfs_client (workstations)

Sets the SELinux boolean for homes on NFS, installs autofs, configures it to read automount maps from IPA (`ipa-client-automount`; the marker is SSSD's automount location), starts it, and confirms it knows both maps. Users' homes appear at `/users/<name>` and team shares at `/teams/<team>` on first access.

### web_proxy (util01)

Squid has two ports: 3128 "standard" and 3129 "allow-list" (an SELinux port label is added for 3129). It allows only CONNECT to port 443, from the workstations only. The standard port denies the social and streaming lists; the allow-list port allows only its list; there is no caching and no TLS interception. The configuration is validated by Squid before it is used; the access log also goes to syslog; the ports are open to the workstation subnet only.

### proxy_client (workstations)

- dnf uses the proxy (by IP).
- Every login shell gets `https_proxy` pointing at its team's port, through `/etc/profile.d/ahmed-proxy.sh`.
- Variables can be changed by the user, so the real enforcement is an **nftables rule matched on the socket owner's UID**: users of allow-list teams are refused on port 3128. Their UIDs

are looked up through SSSD, and the run stops if one cannot be resolved, since the rule would otherwise silently miss them.
- The rule is checked with `nft -c` and loaded by a boot service. It lives in its own table, so firewalld's tables are never touched.

### wireguard_gateway (gw01; `tools.yml` also on control01)

- Refuses to run without peers, with placeholder keys, or without the vault key. It derives the public key from the vault private key and **refuses a key that does not match the recorded public key**, so a wrong vault file cannot silently break every client.
- Installs `wireguard-tools` from a repository definition limited to that one EPEL package over HTTPS, and wstunnel 11.1.0 (pinned, SHA-256 verified, root-owned, SELinux `bin_t`). It loads the kernel module and enables IPv4 forwarding.
- Writes `wg0.conf`. WireGuard's own UDP port is closed to the outside; only wstunnel on TCP 443 is open.
- Per-team firewalld **zones** (one per team range, by source address) and **policies** (one per team, allowing exactly the hosts and ports in `vpn_team_policy`) are written as whole XML files from templates, so a rule removed from the data also disappears from gw01 (D-052). wg0 itself sits in a zone that drops everything else.
- Runs wstunnel as an unprivileged systemd service, then confirms that every configured peer is in the running tunnel, the zone binding, every policy, and the listener.

### hardening (every host, last): CIS Level 1 fixes

| Task file | What it sets |
|---|---|
| `kernel.yml` | Kernel: dmesg restricted, ptrace limited to children, full ASLR, no setuid core dumps; no core dumps stored or allowed. Network (`60-ahmed-network.conf`): no forwarding (except gw01), no redirects sent or accepted, no source routing, strict reverse-path filter, martians logged, ICMP broadcasts and bogus errors ignored, SYN cookies, no IPv6 router advertisements |
| `modules.yml` | 13 kernel modules blocked (rare filesystems, usb-storage, firewire, unused protocols such as sctp, dccp, rds, tipc) |
| `services.yml` | nfs-server, rpcbind and autofs **masked** where the design does not need them |
| `cron.yml` | Root-only crontab and cron directories; empty `cron.allow` and `at.allow` (only root schedules jobs); no deny file |
| `mounts.yml` | `/dev/shm` nodev, nosuid, noexec; **`/tmp` on its own tmpfs** (`tmp.mount`), nodev, nosuid, noexec, because the image has no /tmp partition and a root disk cannot be repartitioned in place |
| `logging.yml` | Persistent, compressed journal; no forwarding to syslog (rsyslog reads the journal itself); local log files 0640; loopback trusted in firewalld |
| `ssh.yml` | Every sshd drop-in mode 0600, and every drop-in (IPA's, the OS's) also says GSSAPI and keyboard-interactive "no", so a future rename cannot bring password logins back (INC-007) |
| `pam.yml` | authselect custom profile `custom/ahmed` (a copy of IPA's sssd profile) with faillock (5 failures, 15 minutes), password history (24, root included) and no empty passwords. pwquality: 14 characters, 4 classes. New accounts: max password age 365, disabled 45 days after expiry, umask 027. 15-minute idle shell timeout; `su` limited to an empty group; dotfiles in local homes tightened |
| `sudo.yml` | `use_pty`, its own log `/var/log/sudo.log` (rotated), re-authentication after 15 minutes |
| `crypto.yml` | System crypto policy `DEFAULT:NO-SSHCBC:NO-SSHWEAKCIPHERS:NO-SSHWEAKMACS:NO-WEAKMAC` (SSH ciphers and MACs only; Kerberos unaffected); sshd restarted |

| | |
|---|---|
| `aide.yml` | AIDE installed; audit tools protected; nightly check to syslog (so to util01); the database rebuilt at the end of every run, because changes made by Ansible are authorized |

### compliance (every host; `compliance.yml`)

Installs `openscap-scanner` and `scap-security-guide` and writes a **tailoring file** from `compliance_exceptions`. It scans with `oscap xccdf eval` against the tailored CIS Level 1 profile, reads the score and the pass/fail counts, compresses the HTML report and XCCDF results (about 19 MB uncompressed per host), and brings them to control01. On util01, `publish.yml` puts them in `/srv/logs/reports/openscap/<fqdn>/` with a `summary.txt` (score against each role's minimum), readable by the security team.

## 5.5 The other playbooks

| Playbook | Purpose |
|---|---|
| `bootstrap.yml` | One time per rebuild, as `ec2-user` with the forwarded laptop key: creates the `ansible` account (password locked), authorizes exactly control01's key, and gives passwordless sudo (validated with visudo) |
| `patch.yml` | Patches a host set two at a time: free-space and failed-service checks, update (or `patch_security_only=true`), `sshd -t`, reboot if `dnf needs-restarting -r` says so (never control01 itself), no newly failed service, a record line in `/var/log/ahmed-patch.log` and syslog, AIDE re-baseline |
| `patch-cycle.yml` + `patch-gate.yml` | The monthly cycle in rings: ws-dev01 (canary); ws-ops01 and util01; gw01 and idm01; control01 last. After each ring the gate checks every host finished and that IPA, SSSD, util01's services and the VPN answer; any failure ends the cycle (D-065) |
| `idm.yml` | FreeIPA (above) |
| `compliance.yml` | The CIS scan (above) |
| `drills/restore.yml` | Runs `ahmed-backup drill` on util01 (D-066) |
| `drills/idm-outage.yml` | Stops IPA, observes the clients (cache, DNS, SSSD state), restarts it, measures each client's recovery |
| `drills/logs-disk-full.yml` | Fills util01's log volume until writes are refused, sends a message from every host, frees the space, measures what arrived |
| `drills/package-rollback.yml` | Downgrades a package on the canary, checks health, patches forward |
| `drills/report.yml` | Shared: appends each drill's result line on util01 and to syslog |

# 6. The scripts

## 6.1 `scripts/lab.sh`: the lab's front door

Everything starts on the laptop with `lab.sh`. It is a thin entry point over three libraries in `scripts/lib/` (D-032, D-041):

| Command | What it does |
|---|---|
| `up [-y]` | Terraform apply (lab layer), then everything `resume` does. About an hour from nothing. `-y` approves the plan automatically. |
| `resume` | On existing infrastructure: load the bootstrap key into the SSH agent; write the SSH config; wait for control01; prepare control01; bootstrap the ansible account; run `site.yml` detached and poll it; reboot control01 if patching needs it; run the compliance scan and the restore drill; run the tests; refresh the VPN client |
| `verify [--full]` | The tests only; `--full` adds a full `site.yml` run that must change nothing |
| `compliance` | Pull on control01, run the CIS scan, publish the results |
| `patch [--security]` | The patch cycle in rings, then the tests |
| `drill <name>` | `restore`, `idm-outage`, `logs-disk-full` or `package-rollback` |
| `status` | Show the instances |
| `down [-y]` | Disconnect the VPN, back up util01 to S3 (it asks before destroying if the backup fails), then `terraform destroy` of the lab layer only |

### The libraries

- **`lib/common.sh`**: paths, region, the key (`~/.ssh/ivolve.pem`, overridable with `LAB_SSH_KEY`), `log`/`warn`/`die`, a tool check, and a lookup of a host's instance ID by its Name tag.
- **`lib/infra.sh`**: `infra_apply` (init and apply), `infra_destroy` (backup first, then destroy), `infra_status`.
- **`lib/control01.sh`**: writes `~/.ssh/ahmed-lab.conf` with two entries, then waits, prepares and reboots control01 as needed.
  - `control01` goes through Instance Connect (`aws ec2-instance-connect open-tunnel`) and always works: it is the break-glass path.
  - `control01-vpn` goes to 10.0.1.10 over the VPN, with the host key stored under the instance ID.
- **`lib/configure.sh`**: `run_on_control01`, plus `configure_hosts`, `run_compliance`, `run_patch_cycle`, `run_drill` and `run_tests`.
  - `run_on_control01` is the key helper. It starts a long command **detached** on control01 (`setsid nohup`, output to `~/run-<name>.log`, exit code to `~/run-<name>.rc`), then follows it with short SSH calls every 20 seconds, printing new `PLAY` and `TASK` lines. A long SSH session through the Instance Connect tunnel had stalled before (INC-002), so long jobs never depend on one connection staying up.

> **Why the login banner repeats during `up`**
> Every poll is a new SSH connection, and sshd prints the company banner to each one. It is cosmetic: the job is running on control01 regardless.

## 6.2 `scripts/control01-setup.sh`

Piped to control01 over SSH, safe to rerun:

1. Installs git and Python, with retries, because the repositories can take a minute after first boot.
2. Clones the repository, or pulls it with `--ff-only`.
3. Creates the venv and installs the pinned `requirements.txt` and collections.
4. Generates the `ansible_ed25519` key if missing, and activates the venv in `.bashrc`.

Everything is inside a function that runs with stdin closed, so no command can swallow the rest of the piped script.

## 6.3 `scripts/vpn-client.sh` (laptop)

| Command | Effect |
|---|---|
| `refresh` | Fetches the gateway's public key and the current public IP, and writes `/etc/wireguard/ahmed.conf`. Run after every rebuild, because the Elastic IP is new each time. |
| `connect` | Starts wstunnel as a user systemd unit (`ahmed-wstunnel`) towards `wss://<gw01>:443`, brings up WireGuard (MTU 1280, route for 10.0.0.0/16 only: a split tunnel), and pings control01 |
| `disconnect`, `status` | Stops both, or shows the state |

It needs sudo for `/etc/wireguard`. During a long `up` the sudo prompt times out, so run `refresh` by hand afterwards.

## 6.4 `scripts/test-lab.sh`: 160 checks

Copied to control01 and run there by `lab.sh verify`. Helpers:

- `check`: a command must succeed.
- `reach`: a TCP port from one host to another must be open or blocked.
- `as_user`/`as_check`: log in as a real IPA user with their key and run something.
- `login` and `sudo_as`: a login or a sudo must be allowed or denied.
- `web`: the proxy's answer for a user and URL: 200 allowed, 403 denied by policy, 000 refused by nftables.
- VPN test peers, one per team, in network namespaces on control01, carried through wstunnel to gw01 like a real client (D-055).

The users' private test keys are decrypted from the vault into `/dev/shm` (memory only) and removed on exit. Sections:

- Inventory and access
- Network policy (default deny, including "must be blocked")
- Baseline configuration (SELinux, firewall, SSH, banners; key-only proven)
- Patch level and patch records
- VPN gateway
- util01 data volume and backups
- Time and identity (FreeIPA, SSSD, IDs, SSHFP, no allow_all, pinned names)
- Logins and sudo per team
- Central logs over TLS, log space guard
- Audit records
- Hardening: crypto policy, AIDE
- OpenSCAP compliance
- Restore drill
- NFS homes

- Storage, quotas, team shares
- Web access per user
- Per-team VPN policy
- Idempotency (with `--full`)

A test tests denial as much as access: "contractor1 login on util01 is denied", "ahmed cannot delete sara's file in the company share", "WireGuard UDP port is closed to the outside". Default-deny designs usually fail exactly there (D-035).

## 6.5 Small helpers

- **`ansible/scripts/vault-password.py`**: Ansible's `vault_password_file`. It reads the SSM parameter with control01's instance role (boto3) and prints it; on error it says what failed, never the value.
- **Scripts on util01**, installed by roles:
  - `ahmed-backup`: backup, restore, verify and drill (6.6).
  - `ahmed-audit-report`: today's audit summary per host.
  - `ahmed-logspace`: the space guard.

## 6.6 `ahmed-backup` (util01)

| Mode | What it does |
|---|---|
| `backup` | `tar` of homes, team shares and logs (with ACLs, extended attributes and SELinux labels; NFS silly-rename files `.nfs*` excluded), streamed by `aws s3 cp -` into `s3://ahmed-infra-backups-<account>/util01/latest.tar.gz`. A tar exit code of 1 ("a file changed while being read", which growing logs cause) is a warning; 2 is a failure. |
| `restore` | Streams the latest backup back and extracts it onto `/`. Used on a new volume. |
| `verify` | Downloads and lists the whole archive |
| `drill` | The restore drill:<br>1. Measures the RPO (time since the data was last safe).<br>2. Plants a marker file in ahmed's home and takes a real, timed backup.<br>3. Deletes the marker and restores just that file. Content, owner by name and SELinux label are checked.<br>4. Restores everything into `/var/tmp/restore-drill` and compares homes and team shares with live (full RTO).<br>5. Writes PASS or FAIL against targets of RPO 24 h, single file 5 min, all data 15 min. |

Owners are stored and restored **by name**. After an IPA reinstall the numbers could differ, and names map files back to the right people (D-056 as built, D-060).

# 7. How it works end to end

*The best way to understand the design is to follow real requests through every layer. These are also the stories to tell in an interview.*

## 7.1 sara logs into ws-ops01 from home

1. **VPN.** On her laptop, wstunnel opens a TLS connection to gw01's public IP on TCP 443. WireGuard's UDP packets travel inside it. gw01's wstunnel hands them to WireGuard on localhost UDP 51820. WireGuard accepts them only because they are signed by sara's peer key, and they arrive on `wg0` from her fixed VPN address in the ops range (10.8.2.x).
2. **gw01's firewall.** Her source address falls in the firewalld zone `vpn-ops`. The forwarding policy for ops allows ws-ops01:22, idm01:22 and util01:22, and nothing else, so the packet is forwarded out of gw01's interface. AWS lets gw01 forward it because its source/destination check is disabled, and gw01's security group egress allows forwarded SSH into the workstation subnet.
3. **AWS security group.** ws-ops01's group `workstation-ops` accepts SSH from `10.8.2.0/24` (and from admin), and from no other team's range.
4. **Replies** go back through the private route table's `10.8.0.0/16 → gw01` route, because there is no NAT: ws-ops01 sees sara's real VPN address.
5. **sshd.** Only public-key authentication is offered. sshd asks SSSD for sara's public key (`sss_ssh_authorizedkeys`, set up by the IPA client). SSSD fetches it from IPA's LDAP, or serves it from its cache.
6. **HBAC.** In the PAM account phase, SSSD checks IPA's HBAC rules: `ops` may log into the ops workstations and the servers host group. A dev user would be refused here even with a valid key.
7. **Home directory.** sara's home is `/users/sara`. autofs on ws-ops01 has IPA's map (`auto.home`, key `*`), so it mounts `util01.ahmed.internal:/export/home/sara` over NFSv4 on TCP 2049. util01 exports only to the workstation addresses; the files belong to UID 455400010, the same number everywhere because IDs are fixed (D-060).
8. **Shell.** `/etc/profile.d/ahmed-proxy.sh` sets her proxy to the standard port; the 15-minute idle timeout and umask 027 apply.
9. **sudo.** `sudo -i` works because IPA's `ops-all` rule covers ws-ops01, delivered by SSSD, with no password (logins are key-only). sudo writes to `/var/log/sudo.log`, and auditd records the sudo command (USER_CMD) and every command she then runs as root (key `privileged`). Both travel to util01, where laila can search them.

## 7.2 ahmed (dev) opens a website

1. His shell's `https_proxy` points at `util01.ahmed.internal:3128` (standard policy, team dev).
2. The workstation's security group allows only util01:3128/3129, so there is no other way out: a direct connection to the internet simply times out.
3. Squid receives `CONNECT www.youtube.com:443`, matches the streaming deny list on the standard port, and answers 403. `CONNECT example.com:443` is allowed; Squid opens the connection through util01's NAT egress, and the TLS session runs end to end between the browser and the site. Squid sees only the domain.
4. If contractor1 (allow-list team) on the same machine pointed his browser at port 3128 to escape his policy, the nftables rule on ws-dev01 rejects any packet to util01:3128 owned by his UID, so he only works on 3129, where only the allow-list passes.

5. Squid's access log is also sent to syslog, so it ends up centrally on util01.

## 7.3 A log line travels to util01

1. A program on ws-dev01 logs to the journal. rsyslog reads it (`imjournal`).
2. rsyslog forwards it over TCP 6514 with TLS, presenting ws-dev01's certificate from the IPA CA (requested and renewed by certmonger) and checking that the server is `util01.ahmed.internal`.
3. util01 accepts only senders with a valid IPA certificate, looks up the sender's address in reverse DNS, and writes `/srv/logs/ws-dev01.ahmed.internal/<program>.log`, group `security`, mode 0640.
4. If util01 is down, the sender keeps the messages in its disk queue and retries every 10 seconds.
5. Audit records follow the same path (auditd's syslog plugin), but are written raw to `audit.log` so the audit tools can read them.

## 7.4 A new host joins the company (enrollment)

1. Terraform creates it with a fixed IP inside the reserved block, a FQDN set by cloud-init, and tags `Role`/`Team`. The dynamic inventory immediately puts it in the right Ansible groups.
2. `bootstrap.yml` creates the `ansible` account; `patch.yml` brings it to the fleet's patch level; `base` applies the baseline.
3. In `idm.yml`, `ipa_client_prepare` creates a one-time password for it on idm01 and registers it in DNS. `ipaclient` enrolls it: it points its resolver at idm01, gets a host keytab, configures SSSD, publishes SSHFP records, and points sshd at SSSD for keys.
4. Host groups, HBAC and sudo rules apply to it as soon as it is in the right host group; `ipa_client_config` resets SSSD's cache when the identity data changed.
5. The log, audit, client, and hardening roles follow. Its compliance scan and the tests prove it.

## 7.5 A rebuild, start to finish (`lab.sh up`)

| Step | What happens | Time (approx.) |
|---|---|---|
| 1 | Terraform apply: 169 resources | 5 min |
| 2 | control01 boots, is prepared (venv, collections, repository, key) | 5 min |
| 3 | `bootstrap.yml`: the ansible account on every host (key forwarded from the laptop once) | 1 min |
| 4 | `site.yml`: patch (RHEL 10.0 image to 10.2), baseline, FreeIPA (the CA install alone takes 5 to 10 minutes), util01 services, logs, audit, clients, VPN, hardening (including AIDE databases) | 40 min |
| 5 | control01 rebooted if patching requires it | 2 min |
| 6 | CIS compliance scan of every host, results on util01 | 5 min |
| 7 | Restore drill (proves the backup that was just restored can be restored again) | 1 min |
| 8 | `test-lab.sh`: 160 checks | 5 min |
| 9 | VPN client refresh (needs your sudo) | - |

util01's data comes back from S3 in step 4: `util_storage` restores the latest backup because the volume group is new.

# 8. Security operations: audit, compliance, patching, drills

## 8.1 Audit (D-063)

auditd runs on every host with the keyed rules listed in section 5.4. Two kinds of record matter most:

- **USER_CMD**: written by sudo itself for every sudo command, with no rule needed.
- **privileged**: every command a real user runs as root, including inside a root shell opened with `sudo -i`, which sudo alone would not show.

Records leave each host through auditd's syslog plugin and the TLS log path, and land raw in `/srv/logs/<fqdn>/audit.log` on util01. The security team reads them there with its sudo rights:

```
sudo ausearch -if /srv/logs/ws-ops01.ahmed.internal/audit.log -k privileged -i     # root
commands by real users
sudo ausearch -if /srv/logs/ws-ops01.ahmed.internal/audit.log -m USER_CMD -i --start today
sudo ausearch -if /srv/logs/ws-dev01.ahmed.internal/audit.log -k sshd --start recent
sudo aureport -if /srv/logs/idm01.ahmed.internal/audit.log --summary -i
cat /srv/logs/reports/audit-$(date +%F).txt     # today's summary, refreshed hourly
```

The rules are deliberately small and explainable rather than the hundreds of vendor STIG rules. They are not locked immutable (`-e 2`), so Ansible can update them without a reboot; CIS Level 1 does not require it.

## 8.2 Compliance: CIS Level 1 at 100% (D-064)

Every host is scanned with OpenSCAP against CIS Level 1 (`cis_server_l1` for servers, `cis_workstation_l1` for workstations), using a tailoring file built from the documented exceptions. The journey:

| Stage | Score | What was fixed |
|---|---|---|
| Baseline (no fixes) | 70 to 74 | 120 distinct failed rules |
| Batch 1 | 92 to 93 | kernel settings, core dumps, kernel modules, unused services masked, cron/at, /dev/shm and /tmp, journald, rsyslog file mode, banner, documented exceptions |
| Batch 2 | 95.6 to 96.2 | network sysctls, SSH settings (and INC-007) |
| Batch 3 | 97.9 to 98.5 | PAM through authselect, password policy, sudo defaults |
| Batch 4 | **100.00** | crypto policy, AIDE, the last umask and GSSAPI details |

Each role's minimum score is now 100 (`compliance_min_score`); a test fails if any host drops below it, so a regression is caught the same day. Fixes live in our own roles. SSG's generated remediation playbooks were rejected: hundreds of unreviewed changes that would fight our roles on every run.

### The documented exceptions

| Rule | Hosts | Reason |
|---|---|---|

| | | |
|---|---|---|
| grub2_password | all | EC2 has no boot console a person could reach; a password only hinders recovery |
| ensure_root_password_configured | all | root stays locked; logins are key-only and break-glass is Instance Connect |
| sshd_limit_user_access | all | IPA HBAC rules decide who may log in where; AllowGroups would duplicate them |
| ensure_journald_and_rsyslog_not_active_together | all | rsyslog reads the journal and forwards it over TLS to util01 |
| package_systemd-journal-remote_installed | all | central logging is rsyslog over TLS, not journal-remote |
| package_bind_removed, package_httpd_removed | idm01 | FreeIPA's DNS runs on bind, its web UI and API on httpd |
| package_squid_removed, service_nfs_disabled | util01 | util01 is the web proxy and the NFS server |
| sysctl_net_ipv4_conf_{all,default}_forwarding | gw01 | gw01 routes VPN traffic into the VPC |
| no_files_or_dirs_unowned_by_user, _ungroupowned | util01 | homes and shares belong to IPA users and groups (resolved by SSSD); the check reads only /etc/passwd and /etc/group |
| service_rpcbind_disabled | util01, workstations | started with the NFS server and client; revisit for NFSv4-only |

Reports: `/srv/logs/reports/openscap/summary.txt` and `<fqdn>/report.html.gz` on util01. Rerun with `scripts/lab.sh compliance`.

## 8.3 Patching cadence (D-065)

| Cycle | When | Command |
|---|---|---|
| Monthly | first Saturday after Red Hat's monthly errata, 02:00 UTC | `scripts/lab.sh patch` |
| Urgent | within 7 days of a Critical or Important security advisory | `scripts/lab.sh patch --security` |
| New or rebuilt hosts | every `up` / `resume` | automatic (first step of site.yml) |

The cycle patches in **rings** to limit the blast radius of a bad update:

1. ws-dev01, the canary.
2. ws-ops01 and util01.
3. gw01 and idm01.
4. control01, which reports a needed reboot instead of rebooting itself.

A gate after each ring checks that every host finished and that IPA, SSSD, util01's services and the VPN still answer. A failure ends the cycle, so the later rings stay on the old, working level. Every run writes a record: the dnf history ID, the number of package changes, whether it rebooted, and the kernel, in `/var/log/ahmed-patch.log` and syslog. Rollback (in `docs/runbooks/patching.md`): `dnf history undo <ID>` or `dnf downgrade`; for a kernel, `grubby --set-default` the previous one; if all else fails, rebuild the host from code.

## 8.4 Backups and restore drills (D-056, D-066)

util01's data is backed up daily at 02:00 by a timer, and before every `lab.sh down`, to the versioned S3 bucket, which keeps old versions for 30 days. On a rebuild the data is restored onto the new volume. The restore drill runs on every `up`/`resume` and measures:

| Measure | Result (2026-11-03) | Target |
|---|---|---|
| RPO: data at risk since it was last safe | 649 s | 24 h (the daily timer) |
| Backup duration | 3 s | - |
| Single-file RTO (a user's deleted file, checked for content, owner and label) | 1 s | 5 min |
| Full-data RTO (98 MB, compared with live) | 2 s | 15 min |

RPO (Recovery Point Objective) is how much work you could lose; RTO (Recovery Time Objective) is how long recovery takes. Disaster recovery for everything else (identity, configuration) is rebuilding from Git: about an hour (`lab.sh up`).

## 8.5 Incident drills (D-067)

| Drill | Result | Follow-up |
|---|---|---|
| idm-outage | SSSD's cache kept users resolving; **DNS failed on clients**; all clients back within about a minute of IPA's restart | D-068: idm01 and util01 pinned in /etc/hosts; the rerun showed DNS OK. An IPA replica stays roadmap item 6. |
| logs-disk-full | With the volume really full, messages were lost: util01 had accepted them over TCP, then could not write them | Space guard below 85%. rsyslog drops on ENOSPC by design, so queues cannot help (tried and reverted). |
| package-rollback | rsync downgraded in 4 s, healthy, patched forward | none needed: the runbook works |
| restore | PASS, every target met | runs on every resume |

Results go to `/srv/logs/reports/drills/` (readable by security) and to `docs/drills/DRILL-00x-*.md`.

# 9. Running the company day to day

*The operations manual. The golden rule: change the data in Git, push, apply from control01, verify with the tests. Never change a server or IPA by hand except to rescue it, and then write the fix into code.*

## 9.1 Starting and ending a working session

```
cd ~/ahmed-infra
eval "$(ssh-agent)"              # the bootstrap key is loaded into it by lab.sh
scripts/lab.sh up                # about an hour; expect 160 passed, 0 failed
scripts/vpn-client.sh refresh    # gw01 has a new public IP after every rebuild (needs sudo)
scripts/vpn-client.sh connect
ssh control01-vpn                # your admin entry; break-glass: ssh control01
```

At the end of a session:

```
git status                       # everything committed and pushed?
terraform -chdir=terraform plan -destroy     # review: only the lab layer (169 resources)
scripts/lab.sh down              # backs up util01 to S3, then destroys the lab
```

**Never interrupt an apply and then destroy**

If `up` is stopped while Terraform is still creating resources, AWS may finish creating something Terraform never recorded in its state (an orphan). The destroy then fails with `DependencyViolation`. Delete the orphan by its `Name` tag and destroy again. Better: let the apply finish, then destroy.

## 9.2 The change workflow (for every ticket)

1. Write the ticket: what, why, who asked. Record a decision in `Decisions.md` if it changes policy.
2. Edit the data file on the laptop; commit; **push**.
3. Apply from control01: `scripts/lab.sh resume` (everything, about 25 minutes), or on control01 a single playbook, for example `cd ~/ahmed-infra && git pull && cd ansible && ansible-playbook playbooks/idm.yml`.
4. Add or adjust checks in `scripts/test-lab.sh` for the new behaviour, including what must be refused.
5. `scripts/lab.sh verify` until everything passes. Close the ticket.

## 9.3 Add a user

1. Get the person's SSH public key; never their private key.
2. In `ansible/inventory/group_vars/role_idm/identity.yml`, add an entry to `ipa_users` with the **next free ID** noted in the file, then update the note:

```
- name: nour
  uid: 455400015            # next free ID; the private group gets the same number
  first: Nour
  last: Developer
  groups: [dev]             # the team decides logins (HBAC), sudo and web policy
  sshpubkey:
    - ssh-ed25519 AAAA... nour@laptop
  # home_quota: { soft: 2g, hard: 3g }      # optional, otherwise the default
  # principalexpiration: "2027-03-31Z"     # for temporary staff
```

3. If the user needs the VPN, add a peer in `role_gateway/main.yml` (their WireGuard public key, team, next free address in the team's range).
4. Commit, push, apply (`resume`, or `idm.yml` then `site.yml` for the home directory and proxy rules).
5. What happens automatically:
   - The IPA account is created with the fixed UID.
   - Group membership, and through it HBAC, sudo and web policy.
   - The home is created on util01 with its quota.
   - The automount entry covers it (key `*`).
   - If the team is an allow-list team, the per-user nftables rule includes the new UID.
6. Test: on control01, `ssh -i <key> nour@10.0.2.10`, then `id`, `pwd` (should be `/users/nour` on NFS) and `sudo -l`. Add checks to `test-lab.sh` (login allowed and denied, sudo, web).

## 9.4 Change a user

| Request | Change |
|---|---|
| Move to another team | Change `groups`. HBAC, sudo, proxy port and nftables follow. SSSD caches are reset automatically when the identity data changes. |
| New or lost SSH key | Replace the key in `sshpubkey`. The old key stops working as soon as SSSD refreshes it. |
| More disk space | `home_quota` on the user, or the default in `role_util/storage.yml`. Limits are only set when they differ. |
| Temporary access | `principalexpiration` |

## 9.5 Remove a user (offboarding)

**Know this before your first offboarding ticket**

The identity role only ever uses `state: present`. Deleting a user from `identity.yml` removes them **at the next rebuild** (IPA is rebuilt from the file), but **not** from a running IPA. For same-day offboarding, the code needs a "disabled or absent users" list handled by the role (`ipauser` with `state: disabled` or `absent`). That is a good first ticket. Until then, the emergency step is `ipa user-disable <name>` on idm01, followed by the code change.

Also: remove the person's VPN peer from `role_gateway/main.yml`, and decide what happens to their home on util01 (archive it in the backup, then remove it).

## 9.6 Add a team

Follow the checklist in `docs/architecture/org-structure.md`:

1. In `identity.yml`: the group (next free GID), its users, a host group, an HBAC rule and a sudo rule (or none).
2. In Terraform: a workstation in `hosts` (a free IP inside the subnet's `10.0.x.8/29`), a VPN range, and the team's workstation security group in `team_workstation_sg`. Plan, review, apply.
3. In `role_gateway/main.yml`: the team's `vpn_team_policy` and peers.
4. In `all.yml`: the team's web policy if it is allow-list, and a team share in `storage.yml`.
5. Run `site.yml`, add tests, verify, record the decision.

## 9.7 Other common tickets

| Ticket | Where |
|---|---|
| Block or allow a website | `role_util/web_policy.yml` (a leading dot matches every subdomain); rerun site.yml |
| Open a network connection | one entry in `terraform/network-policy.tf`; plan shows exactly the two new rules; apply |
| New VPN device | a peer in `role_gateway/main.yml`; the client generates its key pair (`docs/runbooks/vpn-client.md`) |
| Grow storage | a size in `util_storage_volumes`, or a new EBS volume in util01's `extra_volumes` plus `util_storage_disk_count` (`docs/runbooks/storage.md`) |
| Restore a user's file | on util01: `aws s3 cp s3://<bucket>/util01/latest.tar.gz - \| tar -xz -C / export/home/<user>/<file>` (owners come back by name); older versions through S3 versioning |
| Patch | `scripts/lab.sh patch` (or `--security`); read `/var/log/ahmed-patch.log`; rollback in `docs/runbooks/patching.md` |
| Review compliance | `scripts/lab.sh compliance`, then `summary.txt` on util01; a new failed rule is fixed in a role or documented as an exception, never ignored |
| Review audit | as laila on util01: `sudo ausearch -if ...` (section 8.1) and the daily report |
| Rehearse an incident | `scripts/lab.sh drill idm-outage` (or `logs-disk-full`, `package-rollback`, `restore`) |

## 9.8 Troubleshooting checklist

| Symptom | First checks |
|---|---|
| VPN does not connect | `vpn-client.sh status`; `sudo wg show ahmed` (no "latest handshake" means nothing came back); `timeout 5 bash -c '</dev/tcp/<gw01>/443'`; did you `refresh` after the rebuild? Fall back to `ssh control01`. |
| A user cannot log in | Is their key in identity.yml? `id user` on the host (SSSD); `sssctl domain-status ahmed.internal -o`; is there an HBAC rule for the host? `journalctl -u sshd` or `/var/log/secure` centrally on util01. |
| sudo rules missing | `sudo -l -U user` on the host; the identity checksum should have reset SSSD's cache; restart SSSD if it is offline. |
| Home directory missing | `systemctl status autofs`; `automount -m` (maps from IPA); `exportfs -s` on util01; NFS security group and firewall. |

| | |
|---|---|
| Website refused | Which proxy port does the user's shell use (`echo $https_proxy`)? The deny or allow list; Squid's access log on util01. |
| Ansible run hangs | Look at `~/run-site.log` and `~/ansible.log` on control01. Never interrupt dnf. A task over 30 minutes fails on its own. |
| Logs missing on util01 | Sender: `ipa-getcert list -i rsyslog` (MONITORING), `rsyslogd -N1`. util01: `df -h /srv/logs`; the space guard's messages (`ahmed-logspace`). |
| IPA down | On idm01: `ipactl status`, `ipactl start`. Names of idm01 and util01 still resolve (pinned), and cached users keep working. |

## 9.9 Where everything lives (quick reference)

| To change... | Edit |
|---|---|
| Users, groups, HBAC, sudo, IDs, automount | `ansible/inventory/group_vars/role_idm/identity.yml` |
| VPN peers and per-team VPN policy | `ansible/inventory/group_vars/role_gateway/main.yml` |
| Website lists | `ansible/inventory/group_vars/role_util/web_policy.yml` |
| Team to web policy, compliance minimums and exceptions | `ansible/inventory/group_vars/all.yml` |
| Quotas, team shares | `ansible/inventory/group_vars/role_util/storage.yml` |
| Hosts, sizes, disks, IPs | `terraform/variables.tf` (`hosts`) |
| Network connections | `terraform/network-policy.tf` |
| Secrets | `ansible-vault edit <file>` on control01 only; never paste a secret anywhere |

# 10. Every decision, D-001 to D-069

*The full text, with every alternative, is in `Decisions.md`. Here each decision is reduced to what was chosen and the one reason that matters. "Superseded" means a later decision replaced it; the history is kept on purpose, because how a design evolved is part of the story.*

## 10.1 Foundations (D-001 to D-022)

| # | Decision | Chosen | Main reason |
|---|---|---|---|
| 001 | Distribution | RHEL 10 | The platform of the Red Hat lab and most enterprises (dnf, SELinux, IdM); Rocky 10 is the fallback |
| 002 | Scale | 5 hosts, app01 later (6 with gw01) | The company layer is the core; a finished smaller project beats an unfinished large one |
| 003 | Tool split | Terraform builds, Ansible configures | Industry defaults; state for infrastructure, agentless SSH for configuration |
| 004 | Bootstrap SSH key | Import the public half of an existing key | The private key never enters Terraform; replaced by per-user keys in IPA |
| 005 | Instance sizing | superseded by D-025 and D-037 | - |
| 006 | Identity | FreeIPA | Bundles LDAP, Kerberos, DNS, sudo and HBAC; the Linux equivalent of Active Directory |
| 007 | control01 and idm01 separate | Separate hosts | The control plane must be admin-only; IPA must be reachable by every host |
| 008 | Web filtering | Squid with per-team rules | Per-team policy and central logs; HTTPS filtered by domain only (no TLS interception) |
| 009 | Endpoint configuration | Ansible roles by host group | Agentless, same inventory; Satellite is heavy and licensed |
| 010 | Access model | Named admin accounts, no root SSH, control01 admin-only | Every action tied to a person |
| 011 | Region | us-east-1 | Widest availability, lowest prices |
| 012 | Single public subnet | superseded by D-017 | - |
| 013 | Local Terraform state | superseded by D-016 | - |
| 014 | Terraform structure | Local modules (network, security-groups, ec2-instance), one root | One responsibility per module; adding a host or team is one line |
| 015 | Security groups per role | One per role, separate rule resources | New hosts join a role group; no circular references |
| 016 | Terraform state | S3 with native locking (`use_lockfile`) | Recoverable and shareable; no DynamoDB table (completed by D-044) |
| 017 | Network | Tiered private subnets behind a NAT gateway | No server or workstation has a public address; per-tier policy |
| 018 | Bastion host | superseded by D-020, D-023 | - |

| | | | |
|---|---|---|---|
| 019 | Default deny | Groups generated from a flows map; both halves declared; default group emptied | Every opened path is a reviewable one-line change |
| 020 | Entry point as a "gateway" role | gw01 named for its function | The upgrade to a VPN needed no security group redesign |
| 021 | OS image | Looked up by name; AMI changes ignored after creation | Survives image retirement; servers are patched in place, not replaced |
| 022 | Host inventory and instance hardening | One hosts map, static IPs, FQDNs by cloud-init, IMDSv2, encrypted gp3 | FreeIPA needs stable names and addresses; baseline security at no cost |

## 10.2 Access, automation and lifecycle (D-023 to D-045)

| # | Decision | Chosen | Main reason |
|---|---|---|---|
| 023 | Remote access | WireGuard VPN on gw01 | How staff really reach private networks; one key and one IP per user |
| 024 | VPN routing | Routed without NAT; a range per team; split tunnel | Hosts and logs see the real user address, enabling per-team rules and auditing |
| 025 | Instance types and the account plan | idm01 on c7i-flex.large (4 GB) | Free-plan eligible and enough memory for FreeIPA |
| 026 | Bootstrap and break-glass | EC2 Instance Connect Endpoint to control01 | control01 stays private; entry authenticated by AWS credentials |
| 027 | Automation account | Local `ansible` user, key-only, passwordless sudo | Accountable identity separate from humans; works before and without IPA |
| 028 | Where Ansible runs | control01, code pulled from Git | Git is the source of truth; one place applies changes |
| 029 | Inventory | AWS dynamic inventory grouped by tags | Cannot drift; new hosts appear in the right groups |
| 030 | Ansible runtime | venv with pinned versions | Reproducible; the packaged ansible-core is too old |
| 031 | Baseline | base role: time, SELinux enforcing, firewalld, SSH drop-in validated with sshd -t | Independent layers behind the security groups; no self-inflicted lockouts |
| 032 | Lab lifecycle | lab.sh; destroyed between sessions | Repeatable rebuilds; no idle costs; anything not in code is lost |
| 033 | WireGuard role | Repository limited to wireguard-tools over HTTPS; peers as data | Works with default-deny egress; minimal third-party surface |
| 034 | Patch baseline | Fleet-wide update before configuration; reboot only when needed | The 10.0 image vs 10.2 repositories caused drift and a broken systemctl |
| 035 | Verification | test-lab.sh in Git, including "must be blocked" checks | Repeatable; tests denial, where default-deny designs fail |
| 036 | SSH host identity | Host key stored under the instance ID | Replaced servers reuse IPs, never instance IDs |

| | | | |
|---|---|---|---|
| 037 | Control node sizing, patch batching | control01 and gw01 on t3.small; two hosts at a time | A 1 GB control node ran out of memory while patching and broke SSH |
| 038 | VPN firewall zone | Permanent binding (superseded by D-052) | Runtime firewall state is wiped by any reload |
| 039 | VPN endpoint open to any address | Any source | Staff connect from anywhere; WireGuard silently drops unknown senders |
| 040 | VPN transport | WireGuard inside TLS on TCP 443 (wstunnel) | WireGuard UDP was dropped on the admin's network (INC-001) |
| 041 | Script structure | lab.sh over lib/; separate vpn-client.sh | One job per file; a rebuild ends ready to connect |
| 042 | Persistent Terraform layer | Separate state; prevent_destroy | The boundary is structural, not a remembered `-target` flag |
| 043 | Vault password | SSM SecureString, written write-only, read by control01's role | Survives rebuilds; no copy in Git, state or disk |
| 044 | State bucket | Adopted with import; versioned; TLS-only | The one piece not in code; an overwritten state was unrecoverable |
| 045 | Terraform tests | Native tests with mocked providers | Free, fast, no credentials; test the rules the design depends on |

## 10.3 Identity, VPN policy and util01 (D-046 to D-061)

| # | Decision | Chosen | Main reason |
|---|---|---|---|
| 046 | IPA shape | Realm AHMED.INTERNAL, integrated CA, one server | Matches existing names; the CA issues certificates; a replica costs another 4 GB host |
| 047 | DNS | IPA integrated DNS forwarding to the VPC resolver | Automatic records and SSHFP; tradeoff: resolution depends on idm01 (see D-068) |
| 048 | IPA lifecycle | Rebuilt from code every session; identity is data | Same rule as the rest of the lab; adding a team is a data change |
| 049 | IPA secrets | Ansible Vault; clients use one-time passwords | Unattended; clients never see the admin password |
| 050 | IPA automation | freeipa.ansible_freeipa collection, pinned | Idempotent and data-driven |
| 051 | IPA network access | IPA ports from the VPC CIDR, egress to idm01's /32; all six hosts enrolled | Honest boundary without ~72 rules; named, audited admin logins on gw01 |
| 052 | Per-team VPN policy | firewalld zone and policy per team on gw01, plus per-team workstation groups | Two independent layers; whole-file templates remove rules reliably |
| 053 | Ops to servers | Ops range may SSH to idm01 and util01 | No jump host; logs show the person's own address |
| 054 | gw01 itself | SSH from the admin VPN range only | The edge host is admin-only |
| 055 | VPN tests | One synthetic peer per team in network namespaces on control01 | Proves each team's paths with real traffic through the real transport |
| 056 | util01 data | LVM data volume; tar + awscli2 backups to versioned S3; restored on up | Every session is a real restore; no third-party backup tool |

| | | | |
|---|---|---|---|
| 057 | Web access | No direct internet; Squid ports per policy; per-user nftables by UID | The proxy is unavoidable even for root; the policy follows the user, not the machine |
| 058 | NFS homes | NFSv4, sec=sys, workstations only, IPA automount at /users | Works with key-only logins; /home would hide the local accounts |
| 059 | Central logs | rsyslog over TLS with IPA certificates, per host by reverse DNS | Logs carry authentication records; no new PKI; no spoofed directories |
| 060 | Stable IDs | Fixed IPA range and explicit UID/GID | A rebuilt IPA must give everyone the same numbers (NFS, backups) |
| 061 | Storage | Data only on util01; second disk; user and team quotas; team shares with default ACLs | One copy of every file, backed up; workstations disposable |

## 10.4 Roadmap 5 (D-062 to D-069)

| # | Decision | Chosen | Main reason |
|---|---|---|---|
| 062 | Fixed host IPs | Explicit subnet CIDR reservation 10.0.x. 8/29 | The Instance Connect Endpoint took control01's IP (INC-006); covers future interfaces too |
| 063 | Audit | Own keyed rules; records to util01 through the syslog plugin | Explainable, cheap; reuses TLS logging; audisp-remote needs Kerberos and a new port |
| 064 | Compliance | OpenSCAP CIS Level 1; fixes in our roles; exceptions documented; minimum score only rises | The commercial baseline; generated remediation would fight our roles |
| 065 | Patching | Monthly cycle in rings with gates; urgent security cycle | Limits the blast radius; identity patched last |
| 066 | Restore drills | Scripted, measured RPO/RTO against targets, every resume | A backup is only proven by a restore |
| 067 | Incident drills | Three scripted drills with reports | Repeatable; each checks a promise the design makes |
| 068 | Pinned names | idm01 and util01 in /etc/hosts (chosen by Ahmed) | DRILL-001: IPA's DNS was a single point of failure for proxy, NFS and logs |
| 069 | IPA admin access | admin keeps login access to every host, keys only (chosen by Ahmed) | The identity of last resort; passwords are off since INC-007 |

D-065, D-066 and D-067 were proposed on Ahmed's delegation and then approved by him.

# 11. Incidents and drills

*Real problems met while building, with their evidence and root causes (full reports in `docs/incident-reports/`), and the planned drills (`docs/drills/`). Each one changed the code so it cannot happen again.*

## 11.1 INC-001: VPN handshakes dropped

**Symptom:** after the WireGuard gateway was deployed, the admin client could not keep a tunnel up. **Evidence:** gw01 received and answered every handshake, but the answers never reached the client; in a controlled test on the same port and path, plain UDP made the round trip while WireGuard traffic did not. **Root cause (high confidence):** something on the network path recognises WireGuard by its fixed handshake sizes and header bytes and drops it. Changing the port would not help. **Fix:** WireGuard carried inside TLS on TCP 443 by wstunnel (D-040), which looks like HTTPS. **Lesson:** prove a network hypothesis with a controlled comparison before redesigning.

## 11.2 INC-002: `site.yml` froze

**Symptom:** during `lab.sh up` the Ansible run produced no output for over 30 minutes; `lab.sh resume` then finished normally. **Evidence:** package histories showed that no host was running a package task while it was frozen. **Root cause:** not proven; a stalled Instance Connect SSH session is the strong candidate. **Fix:** logging to `~/ansible.log`, SSH keepalives, a 30-minute task timeout, and long jobs run detached on control01 and polled (`run_on_control01`). **Lesson:** say honestly when a root cause is not proven, and make the system robust to the likely causes.

## 11.3 INC-003: firewalld "not running" on a fresh host

A task failed one second after firewalld was started. firewalld is a D-Bus service: systemd marks it "started" as soon as it owns its D-Bus name, before it has loaded its configuration. **Fix:** wait until `firewall-cmd --state` answers. **Lesson:** "the service started" is not "the service is ready".

## 11.4 INC-004: fresh build hung on workstation dnf

The first rebuild after workstations lost direct internet (D-057) hung in `dnf` on ws-dev01: they now needed Squid for packages, but Squid did not exist yet and dnf had no proxy setting. D-057 had been tested on a running lab, never from scratch. **Fix:** `site.yml` patches util01 and builds Squid first, then points workstations' dnf at it by IP. **Lesson:** every change to build order must be tested from nothing.

## 11.5 INC-005: leaked IAM key used for Amazon Bedrock

AWS emailed that the account had accepted an agreement for a model on Amazon Bedrock, which nobody had done. CloudTrail showed a **second IAM user's long-term access key with administrator rights**, leaked outside this project, being used from an unknown machine. It was blocked within minutes. **Impact:** $0, because every model call was throttled. **Actions:** the account was reduced to the root user (MFA) and one IAM user (MFA, one rotated key); every other user, role, admin group and the GitHub OIDC provider were deleted. **Lesson:** long-lived admin keys are the classic cloud breach; use MFA, least privilege and short-lived credentials.

## 11.6 INC-006: control01's IP taken by the Instance Connect Endpoint

A fresh build failed with `InvalidIPAddress.InUse: 10.0.1.10`. The Instance Connect Endpoint, created earlier in the same apply, had been given control01's fixed address by AWS. AWS-managed interfaces take any free address, and nothing reserved the hosts' fixed IPs; earlier builds had passed by luck (roughly 1% per build). **Fix:** explicit subnet CIDR reservations `10.0.x.8/29` (D-062), a validation that every host IP is inside its block, and tests. **Lesson:** a design that works "by luck" fails eventually.

## 11.7 INC-007: SSH offered password logins on every host

**Symptom:** during the CIS work, `sshd -T` showed `kbdinteractiveauthentication yes` everywhere, although our drop-in said `no`. **Root cause:** sshd keeps the **first** value it reads; IPA's `04-ipa.conf` (with `ChallengeResponseAuthentication yes`, an alias) was read before our `10-ahmed-hardening.conf`. The old test tried only the `password` method, so it passed. **Impact:** IPA's `admin` (password in the vault, in the admins group allowed everywhere) could have logged in with its password; no evidence that it happened. **Fix:** the drop-in renamed `01-ahmed-hardening.conf`; every drop-in made to agree; a test that servers offer `publickey` and nothing else. **Lesson:** test what the system actually does (the methods offered), not one path of it.

## 11.8 The drills

| Drill | Scenario | Finding | Change |
|---|---|---|---|
| DRILL-001 idm-outage | All IPA services stopped | SSSD's cache works; DNS for company names failed on clients, so proxy, NFS and log forwarding broke. Recovery took about a minute after the restart. | D-068 pinned names; the rerun showed DNS working; replica is roadmap 6 |
| DRILL-002 logs-disk-full | util01's log volume full | The first run was invalid (2 MB left free). Real runs lost messages: util01 accepts over TCP first, then drops on ENOSPC by design. Queues did not help. | Space guard below 85% (prevents the slow, realistic fill) |
| DRILL-003 package-rollback | "The last update broke a tool on the canary" | rsync downgraded in 4 s, host healthy, patched forward. tmux could not be used: RHEL 10 has only one version of it. | None: the runbook works |
| DRILL-004 restore | A user's file lost; util01's whole data volume lost | RPO 649 s, single file 1 s, full data 2 s; content, owners and labels correct | Runs on every resume |

Two smaller findings along the way: a backup was reported failed once under load, because tar returns 1 when a growing log changes while it is read (treated as a failure under `pipefail`; now a warning). And OpenSCAP reported "unowned files" on util01 because it reads `/etc/passwd`, not SSSD; the files belong to IPA users, so this is a documented exception.

# 12. Lessons and traps

*The things that cost time once and should never cost time again. Most of them are good interview material, because they show real troubleshooting.*

## 12.1 Linux and services

| Trap | What to do |
|---|---|
| sshd keeps the **first** value of a keyword; drop-ins are read in name order | Our drop-in is `01-`; check with `sshd -T`, not by reading files (INC-007) |
| systemd "started" is not "ready" (D-Bus services) | Wait for the service's own readiness check (INC-003) |
| Runtime firewall changes vanish on reload | Always permanent configuration, as whole files (D-038, D-052) |
| XFS cannot enable quotas with a remount | Mount options at mount time; unmount and mount if needed |
| NFSv4 clients do not show POSIX ACLs | Check ACLs on util01, not on the workstation |
| Copying `/etc/skel/.` copies its 0755 mode onto the home | Copy the contents of /etc/skel instead |
| rsyslog drops messages it cannot write; the sender has already handed them over | Keep the volume from filling (space guard) |
| `tar` exits 1 when a file changes while being read | That is a warning; 2 is a failure |
| A partial or killed dnf update can break the system (crypto-policies ahead of SSH) | Never interrupt dnf; patch with enough memory (D-037) |
| An EBS root volume cannot shrink | Replace the host (`terraform -replace`); data lives on util01 anyway |
| `glibc nss-myhostname` answers a host's own name locally | A host resolving **itself** proves nothing about DNS (DRILL-001) |

## 12.2 FreeIPA and SSSD

| Trap | What to do |
|---|---|
| The AWS inventory variable `state` collides with the IPA roles' `state` | Pass `state: present` as a role parameter |
| Enrollment needs full server names; IPA's web server redirects short ones | Use the FQDN (`idm01.ahmed.internal`) |
| A failed enrollment leaves an empty `/etc/ipa/ca.crt` that breaks every retry | Remove it if it is empty and the host is not enrolled |
| IPA refuses a one-time password for a host it still lists as enrolled | Release (disable) the old entry before re-enrolling a rebuilt host |
| SSSD caches "no sudo rules" from before they existed | Reset the cache when the identity data changes (checksum) |
| SSSD can stay offline for minutes after an outage | Restart an offline SSSD on every run |
| A reinstalled IPA gives users new numbers; files are stored by number | Fixed IDs (D-060); backups by name |

| | |
|---|---|
| OpenSCAP reads /etc/passwd, not SSSD | IPA-owned files look "unowned"; documented exception |

## 12.3 Ansible, SSH and shells

| Trap | What to do |
|---|---|
| ssh in a script reads the script's stdin | Use `ssh -n` or `< /dev/null` |
| `pgrep -f` or `grep` match their own command line | Use `[x]yz` patterns. Ansible also logs every command line it runs into `/var/log/messages`, so search a log by its program field, not for a word |
| With pipelining, a running module shows as a bare `python3` | Look at `ps --forest` under the ansible user's sshd session |
| Secrets printed by ad-hoc Ansible end up in logs | Never pipe or print a secret through Ansible ad hoc |
| Long SSH sessions through Instance Connect can stall | Run long jobs detached and poll (`run_on_control01`) |
| Unquoted YAML values containing `": "` become mappings | Quote shell commands that contain a colon and a space |
| Multi-line Jinja in `set_fact` keeps whitespace | Use `{%- -%}` |
| Central log directories are named by FQDN | `/srv/logs/ws-ops01.ahmed.internal/`, not `/srv/logs/ws-ops01/` |

## 12.4 Terraform and AWS

| Trap | What to do |
|---|---|
| AWS-managed interfaces can take a "fixed" IP | Explicit subnet CIDR reservation (INC-006) |
| Deleting a security group hangs while instances still use it | Create the new group and rules, move the instances, delete the old one last |
| Interrupting an apply can leave resources AWS created but state never recorded | Let applies finish; find orphans by tag (`Project=ahmed-infra`) |
| Security group egress also applies to forwarded packets | gw01 needs explicit forwarding egress rules (D-024) |
| Long-lived admin access keys get leaked | MFA, one key, least privilege, rotation (INC-005) |
| Not all instance types are allowed on the AWS Free plan | c7i-flex.large for the 4 GB host (D-025) |

# 13. Interview preparation

*Questions you are likely to be asked about this project, with answers you can give in your own words. Every answer is backed by something in the repository you can show.*

## 13.1 The 60-second pitch

**"Tell me about this project."**

I built the IT environment of a small fictional company on AWS, as its Linux administrator. Terraform builds six RHEL 10 servers in private subnets with default-deny security groups, and Ansible configures everything on them. FreeIPA is the central identity: users, groups, who may log in where, sudo rules, DNS and certificates. Staff connect through a WireGuard VPN with a policy per team, workstations reach the internet only through a Squid proxy with per-user policy, and home directories and team shares come from an NFS server with quotas. Every host sends its logs and audit records to a central log server over TLS. Every host is hardened to the CIS Level 1 benchmark, and an OpenSCAP scan proves it at 100%, with each exception documented. Patching runs in rings with health gates, backups go to S3 and are restore-tested every session, and I run incident drills. The whole lab is destroyed after every session and rebuilt from Git, and 160 automated tests, logging in as real users, prove what is allowed and what is refused. Every decision is documented with its alternatives, and every real incident has a report.

## 13.2 Design questions

**Why FreeIPA and not just local users, or Active Directory?**

Local users do not scale and have no central policy. OpenLDAP would mean assembling Kerberos, DNS, sudo and access rules by hand. Active Directory is the enterprise default, but Windows-based. FreeIPA is Red Hat's own product, the Linux equivalent of AD: LDAP, Kerberos, DNS, a CA, HBAC and sudo rules in one, and RHEL's SSSD is its native client (D-006).

**Why destroy the lab every session?**

It enforces infrastructure as code: anything done by hand is lost, so the code must be complete. It also proves disaster recovery every time (the whole company is rebuilt from Git in about an hour) and removes idle costs, mainly the NAT gateway (D-032, D-048).

**How is the network secured?**

Default deny at every layer. Only TCP 443 on the VPN gateway faces the internet. Security groups are generated from a list of approved flows, with both halves of every connection declared, so anything unlisted is blocked. Host firewalls repeat the boundary, the VPN gateway has a firewalld policy per team, and SSH is key-only. Each boundary has two independent layers, so one mistake does not open a path (chapter 3.3).

**Why does the VPN run inside TLS on port 443?**

WireGuard's UDP traffic was being dropped on my network. I proved it with a controlled test: plain UDP on the same port and path got through, WireGuard did not (INC-001). Changing the port would not help, because the traffic is recognised by its content. wstunnel carries the WireGuard packets inside a TLS WebSocket on 443, which looks like HTTPS. WireGuard inside still does all the encryption and authentication.

#### Why are VPN users not NATed?

So servers see each person's real VPN address. That lets the firewalls tell teams apart, and makes the logs show who did what. It requires disabling the gateway's source/destination check and a return route for the VPN range (D-024).

#### How do you control which websites people can reach?

Workstations have no route to the internet in their security groups, only to the proxy, so even root cannot bypass it. Squid has one port per policy, and an nftables rule on each workstation, matched on the socket owner's UID, makes allow-list users use their own port. HTTPS is filtered by domain only; there is no TLS interception (D-057).

#### Why do workstations have no data disks?

All user and team data lives on util01 and is mounted over NFS at login, like company desktops with network homes. A workstation can be lost or rebuilt without losing anything, users see the same files on any workstation, and there is one copy to back up (D-061).

#### How are secrets handled?

Secrets are encrypted in Git with Ansible Vault. The Vault password is generated by Terraform as an ephemeral value and written to SSM through a write-only argument, so it never appears in state, plan output or any file. Only control01's IAM role can read it, and only through SSM. Hosts enroll in IPA with one-time passwords and never see the admin password (D-043, D-049).

#### What would you change for production?

- A second IPA server (replica) to remove the identity single point of failure.
- Alerting on the log space guard, AIDE and failed drills.
- Golden images to cut rebuild time.
- Short-lived SSH certificates instead of a static automation key.
- Kerberized NFS (krb5p) once users have passwords.
- Multiple Availability Zones, Ansible execution environments, CI with linters, and IAM Identity Center instead of an access key.

## 13.3 Troubleshooting stories (the best interview material)

#### "Tell me about a difficult problem you solved."

**INC-007:** while hardening SSH for the CIS benchmark, I checked the *effective* configuration with `sshd -T` instead of trusting the files, and found every server still offering password logins. sshd keeps the first value it reads, and FreeIPA's drop-in sorted before mine and re-enabled keyboard-interactive. My test had passed because it only tried one authentication method. I renamed my drop-in so it is read first, made every file agree, and wrote a test that checks the methods the server actually offers. Then I wrote an incident report, including the impact: IPA's admin account could have logged in with its password.

**DRILL-002:** I filled the central log disk on purpose and found messages were lost. My first drill was invalid (I had left 2 MB free and the messages fitted), and my first fix (retrying queues) did not work. The server's own log showed why: rsyslog treats a full disk as a permanent error for a message. So the real defence is to keep the disk from filling, and I added a space guard. I kept all three runs in the report, the failed fix included.

**INC-006:** a rebuild failed because AWS gave my control node's fixed IP to an AWS-managed endpoint. Earlier builds had passed by luck. I fixed it with subnet CIDR reservations and a Terraform validation that every fixed IP sits inside the reserved block.

## 13.4 Quick-fire technical questions

| Question | Answer |
|---|---|
| How does a user's SSH key reach the server? | It is stored in IPA; sshd's AuthorizedKeysCommand asks SSSD (`sss_ssh_authorizedkeys`), which fetches or caches it |
| What is HBAC? | IPA's host-based access control: rules saying which users may log into which hosts, checked by SSSD during PAM's account phase |
| Why fixed UIDs? | NFS and backups store numbers; a rebuilt IPA must give everyone the same ones |
| What does idempotent mean? | Running it again changes nothing if the system is already right; here, a second site.yml run reports changed=0 |
| Why separate Terraform states? | So destroying the lab cannot touch what must survive (Vault password, backups, the state bucket) |
| What is a tailoring file? | An OpenSCAP adjustment of a benchmark profile; here, the deliberate exceptions, each with a reason |
| RPO and RTO? | How much data you could lose; how long recovery takes. Measured here at 649 s and 1 to 2 s against 24 h and 5 to 15 min |
| What is IMDSv2 and why require it? | The token-protected metadata service; it blocks SSRF-style theft of instance credentials |
| Why is SELinux enforcing? | A mandatory layer even root cannot override; data areas get the right labels instead of disabling it |
| Why patch in rings? | A bad update hits the canary first, and the identity server only after everything else proved it |
| Why is AIDE rebuilt after every Ansible run? | Ansible is the authorized change path, so the nightly check reports only changes made outside it |
| What does `root_squash` do? | root on an NFS client is mapped to nobody on the server, so it cannot read other users' homes |

## 13.5 Known weaknesses (say them before you are asked)

- One IPA server: identity changes and uncached logins stop while it is down (DRILL-001); a replica is the fix.
- NFS uses `sec=sys`: root on a workstation could impersonate other users' IDs (ops have sudo on ws-ops01); krb5p is the upgrade.
- The per-user web layer can be removed by root on a workstation; the security group still forces everything through the proxy.
- An instantly full log disk still loses messages; the guard handles the realistic slow case.
- Single Availability Zone; no alerting system yet.

# 14. Reference

## 14.1 Command cheat sheet

### Laptop

```
scripts/lab.sh up | resume | verify [--full] | compliance | patch [--security] | drill <name> |
status | down
scripts/vpn-client.sh refresh | connect | status | disconnect
ssh control01-vpn        # admin entry over the VPN
ssh control01            # break-glass through AWS Instance Connect
terraform -chdir=terraform plan          # review before every apply
terraform -chdir=terraform plan -destroy # review before every teardown
terraform -chdir=terraform test          # mocked tests, no AWS needed
```

### control01

```
cd ~/ahmed-infra && git pull && cd ansible
ansible-playbook playbooks/site.yml                  # everything
ansible-playbook playbooks/idm.yml                   # FreeIPA and identity data only
ansible-playbook playbooks/site.yml --limit util01   # one host
ansible all -m ping                                  # who is reachable
ansible-inventory --graph                            # groups from the AWS tags
ansible-vault view inventory/group_vars/role_idm/vault.yml   # secrets: never paste anywhere
tail -f ~/run-site.log ; less ~/ansible.log           # what lab.sh started
```

### Any IPA client

```
id sara                                 # SSSD resolves an IPA user
sudo -l -U sara                         # sudo rules SSSD applies here
sssctl domain-status ahmed.internal -o    # online or offline
sshd -T | grep -E 'authentication|permit' # effective SSH settings
ipa-getcert list                        # certmonger certificates
update-crypto-policies --show
auditctl -l ; ausearch -k privileged -i --start today
firewall-cmd --list-all ; getenforce
```

### idm01

```
ipactl status
kinit admin    # (password from the vault)  then:  ipa user-show sara ; ipa hbacrule-find ; ipa
sudorule-find
```

### util01

```
lvs vg_data ; df -h /export/home /export/teams /srv/logs
xfs_quota -x -c 'report -u -h' /export/home ; xfs_quota -x -c 'report -p -h' /export/teams
exportfs -s ; squid -k check
ls /srv/logs/ ; cat /srv/logs/reports/openscap/summary.txt
/usr/local/sbin/ahmed-backup verify | drill
tail /srv/logs/reports/drills/*.log
```

### gw01

```
wg show wg0 ; systemctl status wstunnel
firewall-cmd --get-active-zones ; firewall-cmd --get-active-policies
```

## 14.2 Repository map

| Path | Contents |
|---|---|
| `README.md` | Overview for visitors |
| `CLAUDE.md`, `PROJECT_HANDOFF.md` | Working rules, and the project's memory (state, lessons, how to continue) |
| `Decisions.md` | D-001 to D-069, each with alternatives and reasons |
| `terraform/`, `terraform/persistent/`, `terraform/bootstrap/` | The three layers (chapter 4) |
| `ansible/playbooks/`, `ansible/roles/` | Playbooks and roles (chapter 5) |
| `ansible/inventory/` | Dynamic inventory and the data files |
| `scripts/` | lab.sh and its libraries, vpn-client.sh, test-lab.sh, control01-setup.sh (chapter 6) |
| `docs/architecture/` | org-structure (teams, access, checklists, security operations), FreeIPA options |
| `docs/runbooks/` | vpn-client, freeipa, storage, patching |
| `docs/incident-reports/`, `docs/drills/` | INC-001 to INC-007, DRILL-001 to DRILL-004 |
| `docs/guide/` | This guide: the HTML source in `src/`, rebuilt with `python3 docs/guide/build.py` |

## 14.3 Glossary

| Term | Meaning |
|---|---|
| Break-glass | An emergency access path that does not depend on the normal one (here: Instance Connect when the VPN fails) |
| CIDR | An address range written as address/prefix, for example 10.0.1.0/24 (256 addresses) |
| Canary | The first, least critical host to receive a change |
| Default deny | Everything is blocked unless explicitly allowed |
| Drift | When a running system no longer matches its code |
| Enrollment | Joining a host to FreeIPA |
| FQDN | Fully qualified domain name, for example util01.ahmed.internal |
| HBAC | Host-based access control: who may log in where (FreeIPA) |
| Idempotent | Running again changes nothing when the state is already right |
| Keytab | A file holding a host's or service's Kerberos keys |
| OTP | One-time password; used here to enroll hosts |
| PTR record | Reverse DNS: address to name |
| RPO / RTO | Recovery point objective (data you could lose) / recovery time objective (time to recover) |
| SSHFP | DNS records holding SSH host key fingerprints |
| SSSD | The client daemon for FreeIPA identities, keys, access rules and caching |
| Split tunnel | Only company traffic (10.0.0.0/16) goes through the VPN |
| Tailoring | Local adjustments to a compliance profile (OpenSCAP) |
| XCCDF / OVAL | The SCAP formats for checklists and results / for how each check is tested |

## 14.4 What is next

The company's admin layer is complete. Remaining build work: **app01** (web and database tier at 10.0.3.10, which activates the dev and dba sudo rules) and, optionally, roadmap item 6 (an IPA replica, alerting, golden images, SSH certificates). Everything else is operations, by tickets, using chapter 9. Before the first ticket, run `scripts/lab.sh up` from nothing: it is the first full build with the hardening, audit and compliance roles, and it should end at 160/160.